European Journal of Operational Research 333 (2026) 117–137

Contents lists available at ScienceDirect

European Journal of Operational Research

journal homepage: www.elsevier.com/locate/eor

Production, Manufacturing, Transportation and Logistics

Integrated trucks assignment and scheduling problem with mixed service mode docks: A Q-learning based adaptive large neighborhood search algorithm

Yueyi Li [a,b], Mehrdad Mohammadi [b], Xiaodong Zhang [a,*], Yunxing Lan [a], Willem van Jaarsveld [b]

[a] Department of Logistics Engineering, School of Traffic and Transportation, Beijing Jiaotong University, ShangYuanCun NO.3, Beijing, 100044, China
[b] Department of Industrial Engineering and Innovation Sciences, Eindhoven University of Technology, Eindhoven, 5600MB, the Netherlands

ARTICLE INFO

Keywords:
Assignment
Truck assignment and scheduling
Mixed service mode
Adaptive large neighborhood search
Q-Learning algorithm

ABSTRACT

Mixed service mode (MSM) docks enhance efficiency by flexibly handling both loading and unloading trucks in warehouses. However, existing research often predefines the number and location of MSM docks prior to planning truck assignment and sequencing. This predefined approach becomes less effective in high-demand systems and increases operational complexity, as warehouse operators must manually test various configurations. This paper addresses this challenge by proposing a new model that integrates dock mode decision, truck assignment, and scheduling, enabling more flexible dock mode arrangements. To solve the complex problem, we introduce a Q-learning-based adaptive large neighborhood search (Q-ALNS) algorithm, which adaptively adjusts dock modes through perturbation operators while simultaneously solving truck assignment and scheduling with destroy and repair operators. The Q-learning mechanism selects these operators based on their performance history and future gains, employing the epsilon-greedy strategy. Comprehensive experimental results and statistical analysis indicate that the Q-ALNS outperforms the benchmark in terms of optimality gap, with an average drop of 12.1%, while maintaining competitive computation efficiency. Compared to the predefined approach, our proposed adaptive strategy reduces tardiness by 22.5% and makespan by 7.6% on average, demonstrating its superiority in improving operational efficiency and supporting demand-driven assignment of MSM docks.

1. Introduction

Docks are critical resources in warehouse operations, serving as the primary interface for inbound and outbound truck handling. However, in most practical settings, the number of available docks is significantly smaller than the number of trucks to be processed (Rijal et al., 2019). Moreover, docks are typically functionally fixed – dedicated exclusively to either loading or unloading. This rigid configuration, combined with temporal imbalances in truck arrivals and departures (Nadi et al., 2021), creates a persistent conflict between fixed dock functions and time-varying demand structures (Neagoe et al., 2021; Wolff et al., 2021). The result is poor resource utilization and long truck queues, particularly in high-throughput warehouses operating under constrained spatial capacity (Jamili et al., 2022; Rijal et al., 2023), highlighting the need for more flexible and adaptive use of dock resources. Recent advances in smart warehousing (Azadeh et al., 2019; Custodio & Machado, 2020; Odeyinka et al., 2023) – such as automated guided vehicles (AGVs), robotic systems, programmable control logic, and real-time warehouse management systems (WMS) – have laid the technical foundation for rethinking static dock functions (Petri et al., 2019). These technologies have already been widely adopted in leading smart warehouses, including JD logistics' "Asia No.1" (JD Corporate Blog, 2020) logistics parks and the focal unmanned facility examined in this study (detailed in Section 5.1). Such systems enable automated equipment to be rapidly reassigned and orchestrated through system-level instructions (He & Prabhu, 2022), supporting the dynamic execution of diverse handling tasks (Kembro & Norrman, 2022). These capabilities have opened up new opportunities for more flexible and adaptive operational strategies in dock management. Among these, the Mixed Service Mode (MSM) (Boysen & Fliedner, 2010) – which allows a single dock to serve both inbound and outbound trucks – has gained increasing attention (Bodnar et al., 2017; Ladier & Alpan, 2016; Rijal et al., 2019; Stephan & Boysen, 2011) as a promising approach to improving dock utilization under fluctuating demand.

Despite its potential, the adoption of MSM in real-world settings faces several practical challenges. First, MSM introduces operational complexity: in traditional warehouses, switching dock functions requires

* Corresponding author.
E-mail address: zhangxd@bjtu.edu.cn (X. Zhang).

https://doi.org/10.1016/j.ejor.2025.12.036
Received 13 September 2024; Accepted 20 December 2025
Available online 24 December 2025
0377-2217/© 2025 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

manual intervention, different equipment, and additional labor coordination (Odeyinka et al., 2023; Stephan & Boysen, 2011). While smart technologies mitigate some of these burdens, the flexibility they enable still incurs measurable costs (Sodiya et al., 2024). Switching a dock's function between inbound and outbound tasks may require system-level adjustments, task re-sequencing, and physical repositioning of equipment – all of which introduce coordination delays. Second, a natural yet critical question in MSM adoption is: how many MSM docks should be used, and where should they be placed? Prior studies typically assume that the number and location of MSM docks are predefined, either as fixed proportions or centralized layouts (Bodnar et al., 2017; Rijal et al., 2019). Such pre-definition may restrict the flexibility of MSM deployment and limit the ability to respond to real-time demand variations (Tutam & White, 2019).

These challenges are also evident in our focal case study – a national-level unmanned warehouse in Shandong, China, equipped with AGVs and robotic systems. While the facility has adopted advanced automation technologies, it still struggles to reassign dock functions efficiently during peak periods. Even when handling directions remain the same, MSM docks require the system to confirm the next task type before dispatching equipment – unlike exclusive-mode docks, where task flows are fixed and uninterrupted. Moreover, the lack of a data-driven method to determine when and where MSM docks should be used forces the warehouse to rely on trial-and-error adjustments, leading to resource underutilization or truck congestion. These observations motivate the need for an integrated scheduling approach that treats dock mode configuration as a decision variable rather than a fixed input. This raises an important theoretical question: how can dock mode decisions be endogenized within the scheduling framework to support real-world operational flexibility?

To address these challenges, we introduce and formalize the *Truck Assignment and Scheduling Problem with Dock Mode Decision* (TASP-DMD), a new integrated scheduling problem that, to the best of our knowledge, has not been explicitly addressed in the literature. In this problem, dock mode decisions (exclusive or MSM), truck-to-dock assignments, and truck sequencing are jointly optimized within a unified model. This framework captures the dynamic interaction between dock configurations and truck allocation, allowing the system to adjust dock functions in response to daily demand fluctuations rather than relying on static predefinitions. To efficiently solve the TASP-DMD and support real-world operational needs, we develop an enhanced Adaptive Large Neighborhood Search (ALNS) algorithm. While ALNS has been widely applied in combinatorial logistics problems, its performance heavily depends on the adaptive operator selection (AOS) mechanism (Pisinger & Ropke, 2019). Existing AOS schemes often rely on score-based rules (Ropke & Pisinger, 2006) and lack responsiveness to the evolving search process (Turkeš et al., 2021). To address this, we incorporate a model-free Q-learning mechanism that enables the algorithm to learn which operator – perturbation or local search – is most effective under different search states. Our approach relaxes the classical perturbation-then-local search structure in meta-heuristic (Fang et al., 2022; Karimi-Mamaghan et al., 2022) and learns a flexible trade-off between exploration and exploitation.

Broadly speaking, this paper addresses the following key research questions:

i) How can the TASP-DMD problem be effectively modeled and solved?
ii) To what extent does incorporating dock mode decisions improve scheduling performance over predefined approaches?
iii) Can Q-learning enhance the performance of ALNS in the TASP-DMD setting, and to what extent?

To answer these questions, the paper makes two main contributions: 1) introducing a dynamic dock mode allocation mechanism integrated with truck assignment and scheduling decisions, enabling flexible decision-making in smart warehouse environments; and 2) proposing a Q-learning-enhanced ALNS (Q-ALNS) algorithm that breaks the classical search structure of meta-heuristic algorithms to improve adaptability and performance.

Beyond warehouses, the same structure of restricted resources with flexible functionality arises in various domains. Airport gates alternate between arrival, departure, and mixed use; planning specifies gate usage, assigns flights, and sequences turnarounds (Jiang et al., 2023; Kim et al., 2023). Hospital operating rooms are reserved for elective, emergency, or mixed demand; planning selects room modes, allocates cases, and sequences surgeries (Majthoub Almoghrabi & Sagnol, 2025; Naderi et al., 2021). These mirror our decision layers (mode configuration, assignment, sequencing) and our Q-ALNS (perturbations for mode changes and local search for assignment and sequence refinement). Comparable opportunities exist in seaport terminals and flexible manufacturing, where resources are configured across functions or product families with changeovers (Bakhshi-Khaniki et al., 2024; Zheng et al., 2024). The transfer requires small adaptations–chiefly a minor reinterpretation of variables and operators and modest constraint or parameter tuning–without altering the underlying optimization structure.

The remainder of the paper is structured as follows. Section 2 presents an overview of existing research related to MSM docks scheduling, ALNS, and Q-learning, then details the contribution of this paper. We formulate the problem in Section 3 and present the proposed Q-ALNS in Section 4. Section 5 designs a set of experiments, and the computational results are reported in Section 6. This analysis leads to conclusions in Section 7.

2. Literature review

2.1. Truck assignment and scheduling for MSM docks

Dock operational decisions focus on efficiently allocating limited dock resources to inbound and outbound trucks over a specific time period. Given the typical imbalance between the number of trucks and available docks, precise timing and sequencing are crucial to minimize waiting times for trucks on-site. From the comprehensive classification by Buijs et al. (2014) and Boysen and Fliedner (2010), it is evident that the assignment and scheduling of trucks are critical operational decisions for docks. Previous research often equated dock scheduling with truck assignment and scheduling, as the number and configuration of docks are typically predetermined at the tactical level. However, with MSM docks, this equivalence no longer holds, as decisions on dock modes are required. A comparative analysis of the most relevant literature on truck assignment and scheduling with MSM docks, as summarized in Table 1, highlights several key points.

In the literature, MSM typically relies on predefined dock configurations (Berghman & Leus, 2015; Hermel et al., 2016; Shakeri et al., 2012; Stephan & Boysen, 2011), where the number and location of MSM docks are fixed prior to the optimization of truck assignments and sequencing. Although some studies (Bodnar et al., 2017; Rijal et al., 2019) have acknowledged the significant impact of dock configurations on MSM efficiency, they still treat the proportion and location of MSM docks as fixed inputs. This sequential decision-making structure limits the overall optimization potential, as subsequent planning is constrained by a predefined layout, and may become less effective under high dock utilization (Rijal et al., 2019). Moreover, this approach requires warehouse operators to manually test various possible configurations, thus increasing the complexity of implementing MSM. Most research focused on cross-docking, aiming for seamless transfers between inbound and outbound trucks with minimal storage buffers (Ladier & Alpan, 2016). A significant gap remains in the application of MSM docks in more general warehouse settings, such as distribution centers (DCs). While some studies focus on optimizing makespan (Shahmardan & Sajadieh, 2020; Shakeri et al., 2012) or addressing internal resource constraints (Hermel et al., 2016; Wolff et al., 2021), most work aims to minimize tardiness and makespan as integrated objectives.

Table 1
Summary of the most related literature.

Publications	Scenarios	MSM	Assignment	Objective	Algorithms
Shakeri et al. (2012)	C	Yes	Dock + Time	Makespan	Two-phase HA
Berghman and Leus (2015)	DC	Yes	Dock + Time	Tardiness, makespan	HA
Hermel et al. (2016)	C	Yes	Dock + Time	Transshipment, makespan	Hierarchical HA
Bodnar et al. (2017)	C	Yes	Time	Transshipment, tardiness	ALNS
Rijal et al. (2019)	C	Yes	Dock + Time	Transshipment, tardiness, storage	ALNS
Xi et al. (2020)	C	Yes	Dock + Time	Penalty cost, number of conflicts	C&CG
Shahmardan and Sajadieh (2020)	C	Yes	Dock + Time	Makespan	HA + RL
Wolff et al. (2021)	C	No	Dock + Time	Resource requirements	CG + HA
Neamatian Monemi et al. (2023)	C	No	Dock + Time	Waiting, makespan, storage, Penalty for early and delay	SVM + BD
This paper	DC	Yes	Mode + Dock + Time	Tardiness, makespan, handling distance	Q-learning + ALNS

MSM: mixed service mode dock; C: cross-docking; HA: heuristic algorithm; DC: distribution center; C&CG: column-and-constraint generation; CG: column generation; ALNS: adaptive large neighborhood search; RL: reinforcement learning; SVM: support vector machine; BD: Benders decomposition.

The impact of MSM dock on internal operational processes is underexplored. Dock operation significantly influences internal resource management (Wolff et al., 2021). However, in most research, MSM docks are often assumed to incur no additional costs, with MSM and exclusive docks considered equivalent. This assumption may overlook the operational complexity introduced by MSM dock. Bodnar et al. (2017), Shakeri et al. (2012) assumed dock flexibility has been incorporated at the design stage, enabling functional switching without additional overhead. In contrast, Rijal et al. (2019) highlight that such functional switching can cause confusion among on-site employees. To address this issue, they advocate for real-time systems to inform staff of the current dock function. Additionally, studies (Bodnar et al., 2017; Rijal et al., 2019) argue that a full deployment of MSM docks may not be necessary and can increase operational complexity. They suggest that a certain proportion of MSM docks is sufficient to achieve efficiency gains. Therefore, it is crucial to incorporate the costs associated with MSM dock selection into the decision-making process, balancing efficient resource allocation with the added operational complexity. This paper addresses these gaps by introducing an integrated decision-making approach that allows for an adaptive dock mode arrangement to address operational complexity and optimize resource utilization. This requires us to develop a new model and solution approach for the problem.

2.2. Original ALNS

ALNS is a meta-heuristic algorithm derived from the Large Neighborhood Search (LNS) introduced by Shaw (1998). ALNS enhances LNS by exploring large neighborhoods through multiple destroy and repair local search operators and an adaptive selection mechanism, enabling the convenient design of neighborhood structures based on decision content (Ropke & Pisinger, 2006). A key element of ALNS is the careful employ of appropriate operators, as well as the integration of an adaptive layer to dynamically select these operators during the search process (Windras Mara et al., 2022).

Rigorous analysis and test of operators (among multiple operators) are crucial to the success of ALNS (Karimi-Mamaghan et al., 2022; Voigt, 2024). The flexibility of the ALNS framework often leads researchers to include a large number of operators, increasing the chances of finding better solutions. However, an unnecessarily large set of operators, possibly including inappropriate operators, can reduce algorithm efficiency. Scholars (Turkeš et al., 2021; Windras Mara et al., 2022) have accordingly emphasized the importance of thorough operator analysis. In a related study, Voigt (2024) tested and ranked search operators for vehicle routing problems (VRP), showing that performance differences among operators exist. Pre-set search operators may perform well on similar instances but poorly on others with different characteristics. This paper underscores the necessity of evaluating and filtering operators tailored to specific problems. In the context of TASP-DMD problems, the abundance of operators will be carefully filtered and selected based on their expected effectiveness.

In ALNS, the classical AOS mechanisms follow the roulette wheel principle as Ropke and Pisinger (2006) and do not take the status of the search process and the operators' history of performance into account (Turkeš et al., 2021). For example, Bodnar et al. (2017) assigned three types of scores to operators based on outcomes such as yielding a solution better than the global best, yielding a solution better than the current best, or no improvement. These scores are then used to update the weights of each operator accordingly. Similarly, Rijal et al. (2019) employed a similar AOS mechanism for their study to handle dock allocation alongside truck scheduling. However, the score-based method often overlooks differences in operator performance, as it simply classifies performance levels and is often user-defined (Johnn et al., 2023). Studies like Turkeš et al. (2021) have pointed out that simple score-based AOS mechanisms used in most ALNS offer limited improvement to the algorithm's performance. Developing effective AOS mechanisms remains an open challenge. In this regard, learning-based methods like machine learning techniques have emerged as promising alternatives (Karimi-Mamaghan et al., 2022). Compared to score-based methods, learning-based methods can capture and exploit richer information related to operator selection, leading to better overall performance. Johnn et al. (2023) introduced a deep RL-based operator selection mechanism for the capacitated VRP, demonstrating that learning-based mechanisms outperform both random and score-based methods. As shown in Table 1, recent relative research has begun using machine learning techniques to improve (meta-)heuristic or exact algorithms. Neamatian Monemi and Gelareh (2023) used support vector machines to identify effective objective functions in different Benders decomposition stages, demonstrating the framework's effectiveness. Combining reinforcement learning (RL) and other machine learning techniques with heuristic-based algorithms requires further research.

2.3. Integration of Q-learning into ALNS

Q-learning is a widely used RL algorithm that enables agents to learn optimal policies through interactions with the environment. Its model-free nature makes it suitable for complex and dynamic settings where constructing an accurate model is challenging or infeasible. The integration of Q-learning with ALNS can be approached from two main perspectives. The first is to leverage the complementary strengths of both methods in addressing dynamic optimization problems (Zhang et al., 2023). The second approach focuses on optimizing specific components of the ALNS algorithm using Q-learning, particularly the AOS mechanism. In this paper, we focus on the latter, specifically enhancing the AOS mechanism within ALNS to improve the overall search process.

When integrated into AOS mechanism, Q-learning can extract useful information from the search process (e.g., the sequence of operators, the amount of improvement they led to, interdependence between operators, etc.) and use this information to guide subsequent searches (Johnn et al., 2023, 2024), leading to more adaptive and efficient outcomes. Boualamia et al. (2023) incorporated Q-learning into ALNS's

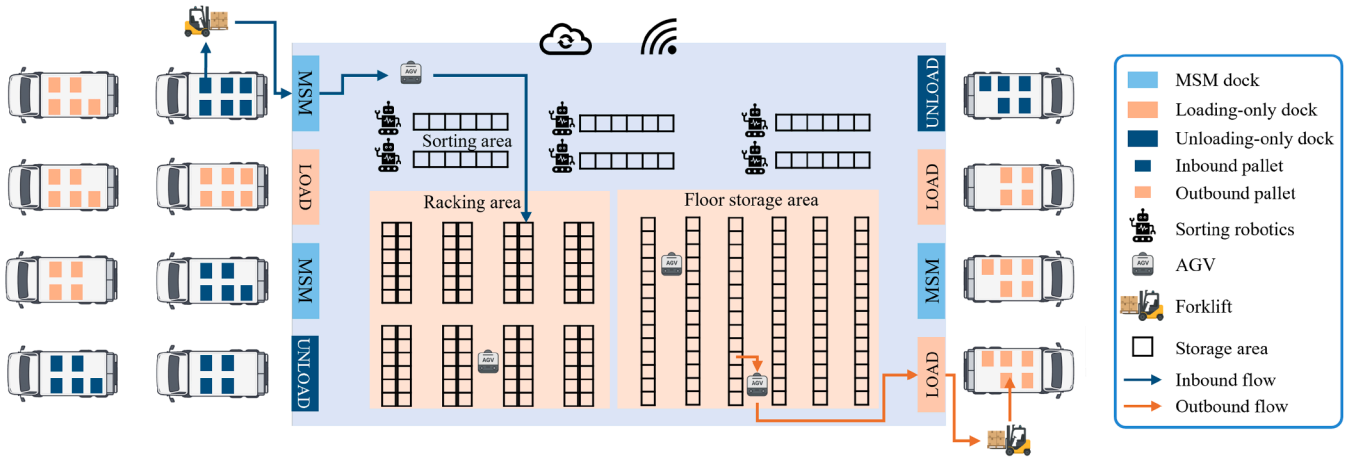


Fig. 1. Truck assignment and scheduling problem with dock mode decision in a UDC.

AOS mechanism addressing a capacitated VRP. After reaching a local minimum, the epsilon-greedy strategy was used to select new operators based on Q-values. Wang et al. (2024) modified the adaptive layer of ALNS using the DQN mechanism to solve a pickup and delivery problem with time windows. The DQN-based AOS mechanism adaptively selects 20 operators to solve the problem. These studies generally focus on the selection of local search operators, without validating the efficiency of the operators through the OF process. Compared to traditional ALNS, the improvements in solution quality and CPU time are limited.

In a related field, Karimi-Mamaghan et al. (2023) and Karimi Mamaghan et al. (2024) combined Q-learning with the iterated greedy algorithm. They proposed a value-based AOS mechanism, where Q-learning is used to select perturbation operators. Their approach demonstrates improvements in both solution quality and convergence rate, serving as an important inspiration for this work. In our TASP-DMD problem, changes in dock mode offer a greater optimization potential for truck assignment and sequencing. To address this, we extend the original ALNS framework by introducing a perturbation mechanism that is triggered when local search fails to find better solutions over extended search periods, thereby expanding the search space. Fang et al. (2022) also incorporated both perturbation and local search operators into the ALNS framework. However, their approach follows a fixed perturbation-then-local-search structure, which is a common characteristic of most hybrid heuristics. This fixed structure may limit the algorithm's flexibility and its ability to adapt to dynamic problem features.

To the best of our knowledge, no research has yet utilized Q-learning to guide both the perturbation and local search processes within a complete algorithmic framework. We aim to bridge these gaps by introducing greater flexibility into the search process, allowing Q-learning to guide both perturbation and local search phases, thus overcoming the fixed search structure. This approach not only selects promising operators but also dynamically adjusts the sequence and strategy of applying perturbation and local search operators, thereby balancing exploration and exploitation.

2.4. Contributions of this paper

This paper contributes to the literature through the following aspects:

- We introduce a novel integrated model for MSM-based operations. We formulate an integrated model for the TASP-DMD problem, which simultaneously optimizes dock mode arrangement, truck assignment, and scheduling. Unlike existing approaches that rely on predefined MSM configurations, our model captures the dynamic interaction between dock modes and truck scheduling, thereby enabling demand-driven, flexible operational decisions in modern smart warehouses. In addition, the model supports warehouse operators by informing decisions at both the facility design stage – helping to determine whether to adopt MSM docks – and the daily operational stage – guiding how to effectively utilize MSM docks.
- We develop an enhanced ALNS algorithm that incorporates Q-learning (Q-ALNS) to improve the operator selection mechanism. First, we employ a pairwise operator comparison mechanism to identify better-performing operators, which improves the algorithm's performance by focusing on the most effective operators. Second, unlike the classical AOS mechanism, our approach takes the operators' history of performance and the status of the search process into account. Additionally, our approach relaxes the traditional search processes, which follow a fixed perturbation-then-local search structure, and learns a flexible trade-off between exploration and exploitation in metaheuristics. With this dual learning process, our Q-ALNS algorithm achieves superior performance compared to benchmark methods while maintaining computational efficiency.

3. Problem description and formulation

3.1. Problem description

We consider an unmanned distribution center (UDC) that manages the allocation and scheduling of a set of trucks T, consisting of inbound trucks $i \in I$ and outbound trucks $j \in O$. As illustrated in Fig. 1, each dock $k \in D$ can operate in one of the three modes: loading-only, unloading-only, or MSM. Inbound trucks $i \in I$ can be assigned to either the unloading-only or MSM docks. Outbound trucks $j \in O$ can be allocated to either the loading-only or MSM docks. The dock mode assignment is part of the decision-making process and is not spatially fixed; inbound and outbound flows may occur on either side of the warehouse.

Each inbound truck carries a known number of pallets, which are unloaded by forklifts and transferred to a designated location within the warehouse by AGVs. Outbound trucks are loaded with pallets retrieved from the corresponding location by AGVs and then handled by forklifts at the assigned docks. The storage location of each pallet, as well as the pallet type, is known in advance. All equipment operates according to the decisions and instructions provided by the WMS, with tasks executed and status feedback transmitted through IoT devices and wireless networks, ensuring seamless integration and real-time coordination across the warehouse operations. In this environment, dock functionality can be adjusted based on real-time WMS instructions (Rijal et al., 2019) without requiring physical layout changes, as this flexibility was incorporated during the design phase (Bodnar et al., 2017; Shakeri et al., 2012).

Trucks arrive according to predetermined time windows (r_i, d_i), where r_i is the arrival time and d_i is the expected departure time. The lower bound r_i is fixed, while the upper bound may be extended depending on system load. Then the truck delay accounts for any extensions beyond the expected departure time d_i. Most research does not consider the impact of internal transport effort, focusing more on the completion time of tasks at the truck end. However, in resource-limited situations, the efficiency of internal transport is a crucial factor for overall warehouse efficiency (Wolff et al., 2021). To model internal transport effort, we incorporate the AGV travel distance between docks and storage locations into the objective function, capturing the impact of dock mode decisions on overall transport efficiency. While detailed AGV routing models exist, we focus on aggregate travel distances to balance model complexity and computational tractability.

In addition, MSM docks require a switching time τ due to changes in both information flow and equipment repositioning. Based on observation of a real-world UDC and discussion with onsite manager, deployment MSM dock involves both information-level adjustments and physical reconfiguration of equipment such as AGVs and forklifts. This process necessitates a buffer period to verify the next task type and reassign resources, even when consecutive trucks are of the same type. From a scheduling perspective, this switching time is analogous to setup time in unrelated parallel machine scheduling (Xie et al., 2025), where machines (i.e., docks) require reconfiguration between different job types. While the actual switching time may be negligible in cases where task types remain consistent, the verification process itself introduces nonzero overhead. To reflect the operational complexity and ensure model tractability, we define τ explicitly as the *dock reconfiguration time*, i.e., the additional setup incurred when an MSM dock switches between tasks. In our model, τ is treated as a fixed constant for all MSM docks, representing a buffer for safety checks and control coordination. A sensitivity analysis of this factor is reported in Section 6.2.4 to evaluate its impact under different operational settings. Furthermore, an extended model is provided in Appendix B, where τ becomes flow-dependent and is incurred only when the commodity type differs between consecutive tasks, offering a more flexible and realistic formulation for future applications.

In the TASP-DMD problem, scheduling decisions encompass the mode of the dock, truck-to-dock assignment, and truck scheduling. The optimization objectives include minimizing truck delay, the maximum completion time, and the weighted distance of cargo handling. Several assumptions have been made in this paper based on practical cases and experience: 1) Each dock is designed to be equipped with sufficient forklifts, and the travel paths of the forklifts are fixed, simplifying loading and unloading time to the time per pallet, denoted as t_i; 2) Trucks can only begin operations after their scheduled arrival time; 3) A truck can leave the docking position only after completing its loading or unloading tasks; 4) All cargo is palletized, and storage capacity is assumed to be sufficient and non-restrictive; 5) No ad-hoc truck loading or unloading tasks are included; and 6) Compatibility between docking positions and truck types is not considered.

3.2. Notations

Before proposing the mathematical model of the problem, parameters and decision variables are listed in Table 2.

3.3. Truck assignment and scheduling model with dock mode decision

This section constructs a multi-objective mixed-integer programming model that simultaneously optimizes dock mode, truck assignment, and truck scheduling. The optimization objectives are to minimize dock operation delay time (referred to as tardiness), the maximum completion time (referred to as makespan), and the distance of cargo handling (referred to as distance).

The tardiness refers to the sum of the delays of all trucks' operations compared to their scheduled departure times. The expression for the first objective function is shown in Eq. (1). Here, δ_i represents the delay time for truck i, defined as Eq. (2).

$$f_1 = \min \sum_{i \in T} \delta_i \tag{1}$$

$$\delta_i = \max\{0, e_i - d_i\}, \forall i \in T \tag{2}$$

The makespan refers to the latest time by which all loading and unloading tasks are completed. The expression for the second objective function is presented in Eq. (3). Here, H_i represents the completion time for truck i, defined as Eq. (4). The first term accounts for the forklift's loading and unloading time, the second term for inner warehouse handling time, and the third term for the switching time required at mixed-mode docks. The switching time is introduced to address the operational complexity of MSM mode docks, as directly modeling this complexity is challenging. The sum includes all these times only when both x_{ik} and μ_k are set to 1.

$$f_2 = \min \max\{H_i \mid i \in T\} \tag{3}$$

$$H_i = t_i + \sum_{k \in D} \frac{m_{ik}}{v} x_{ik} + \sum_{k \in D} \tau \mu_k x_{ik} \tag{4}$$

Assigning trucks to different docking positions results in varying distances for internal handling. The third objective function aims to minimize the total distance for handling. The expression for this objective function is detailed in Eq. (5). Detailing the scheduling of AGVs would significantly increase the complexity of the problem. Therefore, we adopt the Euclidean distance between dock positions and storage locations. Since the number of pallets on each truck and their corresponding storage or retrieval locations within the warehouse are known in advance, the parameter m_{ik}, representing the inner handling distance between truck i and dock k, can be determined once the truck is assigned to a specific dock. This parameter is pre-processed and maintained in the form of a distance matrix between trucks and docks. Future research will address detailed scheduling and further extend this parameter.

$$f_3 = \min \sum_{i \in T} \sum_{k \in D} m_{ik} x_{ik} \tag{5}$$

The first category of constraints pertains to the allocation of dock modes and positions. Eq. (6) ensures that each dock is designated for only one service type: unloading-only (λ_k), loading-only (ρ_k), or mixed-mode (μ_k). Eqs. (7) and (8) specify that each truck is assigned to only one dock. Eq. (9) ensures that unloading trucks are assigned only to docks that are capable of handling unloading, specifically exclusive unloading docks or MSM docks. Eq. (10) applies a similar logic to outbound trucks. Eq. (11) ensures that trucks i (unloading) and j (loading) are assigned to the same dock only when $\mu_k = 1$, providing a linear representation of the constraint $x_{ik} y_{jk} \le \mu_k$.

$$\lambda_k + \mu_k + \rho_k = 1, \forall k \in D \tag{6}$$

$$\sum_{k \in D} x_{ik} = 1, \forall i \in I \tag{7}$$

$$\sum_{k \in D} y_{jk} = 1, \forall j \in O \tag{8}$$

$$\sum_{k \in D} (\lambda_k x_{ik} + \mu_k x_{ik}) = 1, \forall i \in I \tag{9}$$

$$\sum_{k \in D} (\rho_k y_{jk} + \mu_k y_{jk}) = 1, \forall j \in O \tag{10}$$

$$x_{ik} + y_{jk} - 1 \le \mu_k, \forall i \in I, j \in O, k \in D \tag{11}$$

The second category of constraints deals with the timing of truck loading and unloading operations. Eqs. (12) and (14) represent the time window constraints. Eq. (13) defines the unloading/loadingcompletion

Table 2
Notation for TASP-DMD problem.

Symbol	Notation Definition
Sets	
I	Set of unloading trucks, indexed by $i \in I$.
O	Set of loading trucks, indexed by $j \in O$.
T	Set of all trucks, $T = I \cup O$.
D	Set of docks, indexed by $k \in D$.
Parameters	
r_i	Scheduled arrival time of truck i, $\forall i \in T$.
d_i	Expected departure time of truck i, $\forall i \in T$.
t_i	Loading and unloading time for truck i.
τ	Switching time required before each truck starts loading and unloading operations at mixed-mode docks.
m_{ik}	Total inner handling distance for truck i at dock k.
v	Travel speed of the handling equipment provided at each dock, m/min.
$L_{\max}$	The maximum allowed completion time for any truck.
M	A sufficiently large constant (Big-M) used for constraint linearization.
Decision Variables	
x_{ik}	is 1 if truck i designated for unloading is assigned to operate at dock k, and is 0 otherwise
y_{jk}	is 1 if truck j designated for loading is assigned to operate at dock k, and is 0 otherwise
λ_k	is 1 if dock k is exclusively for unloading, and is 0 otherwise
ρ_k	is 1 if dock k is exclusively for loading, and is 0 otherwise
μ_k	is 1 if dock k is for mixed-mode, and is 0 otherwise
a_i	Start time of operations for truck i, $\forall i \in T$.
z_{ijk}	is 1 if truck i and truck j are assigned to dock k, and i is the preceding truck in the sequence before j, and is 0 otherwise
Auxiliary Variables	
δ_i	Total delay time for truck i.
e_i	End time of operations for truck i, $\forall i \in T$.
H_i	Completion time of all truck i's loading or unloading tasks.
u_{ijk}	Auxiliary binary variable equal to 1 if truck i immediately precedes truck j on dock k, 0 otherwise.
w_{ik}	Auxiliary binary variable satisfying $w_{ik} = \mu_k x_{ik}$.
$H_{\max}$	Auxiliary continuous variable representing $\max_{i \in T} H_i$.

time for each truck i.

$$a_i \geq r_i, \forall i \in T \tag{12}$$
$$e_i = a_i + H_i, \forall i \in T \tag{13}$$
$$e_i \leq L_{\max}, \forall i \in T \tag{14}$$

The third category of constraints pertains to the sequencing or ordering of trucks. Eqs. (15)–(17) define the sequencing of trucks assigned to the same dock and ensure the uniqueness of order. Eq. (18) defines the sequential relationship in time for trucks serviced at each docking position.

$$z_{ijk} + z_{jik} = x_{ik}x_{jk}, \forall i, j \in I, k \in D \tag{15}$$
$$z_{ijk} + z_{jik} = y_{ik}y_{jk}, \forall i, j \in O, k \in D \tag{16}$$
$$z_{ijk} + z_{jik} = x_{ik}y_{jk}, \forall i \in I, j \in O, k \in D \tag{17}$$
$$a_j + M(1 - z_{ijk}) \geq e_i, \forall i, j \in T, k \in D \tag{18}$$

The final category of constraints pertains to the types of variables.

$$x_{ik}, y_{jk} \in \{0, 1\}, \forall i \in I, j \in O, k \in D \tag{19}$$
$$\lambda_k, \mu_k, \rho_k \in \{0, 1\}, \forall k \in D \tag{20}$$
$$z_{ijk} \in \{0, 1\}, \forall i, j \in T, k \in D \tag{21}$$
$$a_i \geq 0, \forall i \in T \tag{22}$$

3.4. Model linearization

The proposed model contains several nonlinear terms, which can be linearized by introducing auxiliary variables and linear constraints.

Eq. (2) can be expressed using the following linear constraints:

$$\delta_i \geq e_i - d_i, \forall i \in T \tag{23}$$
$$\delta_i \geq 0, \forall i \in T \tag{24}$$

By introducing an auxiliary variable $w_{ik} = \mu_k x_{ik}$, the following constraints are added:

$$w_{ik} \leq \mu_k, \forall i \in I, \forall k \in D \tag{25}$$
$$w_{ik} \leq x_{ik}, \forall i \in I, \forall k \in D \tag{26}$$
$$w_{ik} \geq \mu_k + x_{ik} - 1, \forall i \in I, \forall k \in D \tag{27}$$
$$w_{ik} \in \{0, 1\} \tag{28}$$

The updated Eq. (4) then becomes Eq. (29), where the nonlinear product term is replaced by a linear expression.

$$H_i = t_i + \sum_k \frac{m_{ik}}{v} x_{ik} + \sum_k \tau w_{ik} \tag{29}$$

Introducing an auxiliary variable $H_{\max}$, we enforce the following constraints:

$$H_{\max} \geq H_i, \quad \forall i \in T \tag{30}$$

Then we can reformulate the minimization of the maximum term into a linear formulation:

$$f_2 = \min H_{\max} \tag{31}$$

For constraint Eq. (15), we can decompose multiplicative terms into linear constraints. Eqs. (32)–(34) indicate that z_{ijk} can only be 1 if trucks i and j are assigned to the same dock. Eq. (35) addresses the potential confusion in the assignment of truck sequencing variables, stipulating that if truck i precedes truck j at the same dock, then j cannot precede i at that dock. Eqs. (16) and (17) can be linearized the same way.

$$z_{ijk} \leq x_{ik} \tag{32}$$
$$z_{ijk} \leq x_{jk} \tag{33}$$
$$z_{ijk} \geq x_{ik} + x_{jk} - 1 \tag{34}$$
$$z_{ijk} + z_{jik} \leq 1 \tag{35}$$

Additionally, the three-objective function can be handled using the ϵ-constraint method, where f_1 (tardiness) is set as the primary objective while f_2 (distance) and f_3 (makespan) are converted into constraints. To generate a representative set of Pareto solutions, we first solve single-objective problems to obtain the minimum and maximum values of f_2 and f_3 for each instance. These intervals are then discretized using fixed step sizes (1.0 for both objectives) to form a grid of (ϵ_2, ϵ_3) pairs. For each pair of thresholds (ϵ_2, ϵ_3),we solve a subproblem that minimizes f_1 while

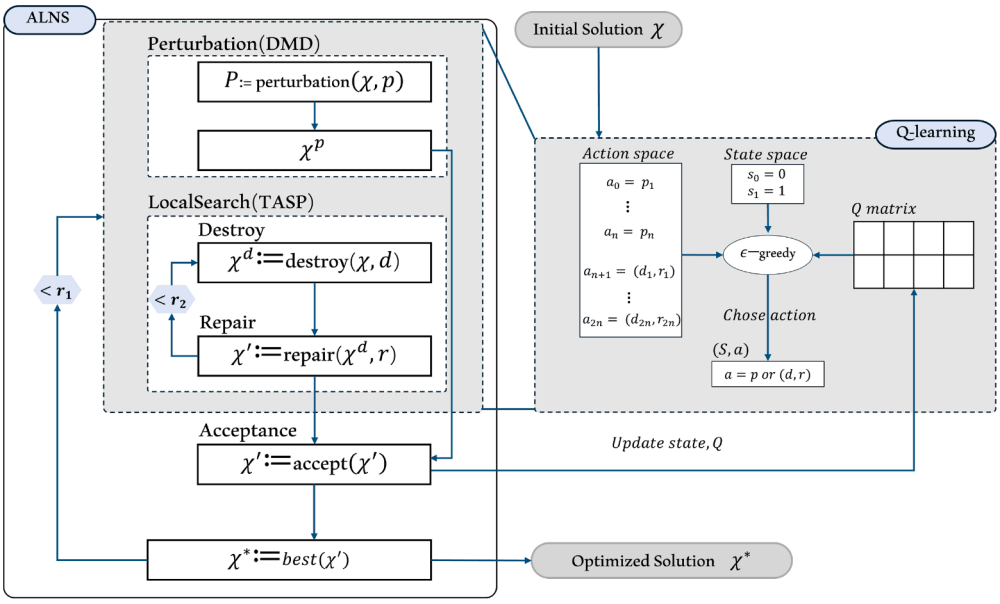


Fig. 2. Flowchart of the Q-ALNS.

enforcing the constraints $f_2 \leq \epsilon_2$ and $f_3 \leq \epsilon_3$. All solutions are filtered to remove dominated ones, yielding a representative set of non-dominated solutions on the Pareto front. Although the model still contains a few convex quadratic constraints, Gurobi's built-in convex MIQCP solver can handle these constraints directly and guarantees a globally optimal solution.

Testing with the instances in Table 3, Gurobi successfully obtained a set of Pareto-optimal solutions for the first three small-scale instances. However, in the remaining six instances, Gurobi failed to return a solution within the given time limit (3600s). The integrated dock assignment and vehicle scheduling problem is already NP-hard (Rijal et al., 2019), and incorporating dock mode decisions further increases the computational complexity. Therefore, more efficient algorithms are required to address real-world applications.

4. Proposed ALNS with a Q-learning-based AOS: Q-ALNS

To efficiently solve the model presented in Section 3, this study proposes an improved ALNS algorithm integrated with a Q-learning-based AOS mechanism, named Q-ALNS. Fig. 2 presents the detailed structure and operational logic of the proposed Q-ALNS algorithm. Additionally, Algorithm 1 provides the detailed Pseudo-code. The overall framework consists of two main interacting components: an ALNS procedure (left side) and a Q-learning-based AOS mechanism (right side).

The algorithm begins with an initial feasible solution χ. The ALNS component iteratively explores solutions through two distinct stages: the perturbation phase (Dock mode decision, DMD) and the local search phase (Truck assignment and scheduling problem, TASP). In the perturbation phase, a dock mode perturbation operator p is applied to the current solution χ, generating a modified solution χ^p. In the local search phase, a destroy operator d removes a subset of truck assignments and scheduling decisions from solution χ, resulting in an incomplete solution χ^d, while a repair operator r reconstructs χ^d into a feasible candidate solution χ'.

The right side of Fig. 2 highlights the Q-learning-based AOS mechanism, designed to adaptively guide operator selection based on the state and history of solution quality improvements. Operator selections, including the perturbation operator and paired destroy-repair operators, constitute the action space of the Q-learning algorithm. Unlike supervised learning, Q-learning does not involve a traditional split of data into training and test sets. Instead, we stabilize the learning process by designating the first 5% of the Q-ALNS's iterations as a learning loop. During this phase, the Q-table is updated but not exploited for operator selection, which helps prevent premature bias toward operators with high initial rewards. Then actions are chosen via an ϵ-greedy strategy, balancing exploration and exploitation by utilizing a Q-table updated according to observed improvements. This strategy uses the Q-table to choose actions with the highest credit or explore new actions, preventing the algorithm from getting stuck in local optima. Each action receives an immediate reward corresponding to its performance, enabling the Q-learning mechanism to iteratively refine its selection strategy.

The main improvements of the proposed Q-ALNS over the original ALNS are as follows:

1) The main loop is extended into a two-phase loop. We add a perturbation phase into ALNS framework, in which perturbation operators are employed to achieve diverse neighborhood searches for dock mode decisions. Since dock mode manipulations represent *high-level, severe changes* that can drastically reshape the feasible solution space, they are modeled as perturbations to shake the solution and escape local optima. Subsequently, the local search phase uses destroy and repair local search operators, which correspond to *low-level, mild adjustments* of truck assignment and scheduling under the given dock mode. This approach allows for the simultaneous optimization of dock mode, truck assignment, and scheduling.

2) The Q-learning algorithm is integrated into the AOS mechanism, which bases its decisions on the search state and history of performance. Unlike score-based methods, Q-learning also accounts for future gains, encouraging long-term benefits.

3) We further introduce an independent relaxation of the conventional fixed sequence of perturbation and local search phases. Instead of always executing perturbation followed by local search, the algorithm can dynamically decide which phase to apply based on the search state. This relaxation increases the flexibility and adaptiveness of the framework. Importantly, it also ensures that the effect of dock mode perturbations is not diminished by being immediately overridden by local search; rather, their influence is adaptively reinforced whenever they prove beneficial.

4) Since we are dealing with a multi-objective model, our algorithm updates solutions by maintaining and updating a Pareto set of solutions. A new solution is accepted under one of the following four conditions: when the new Pareto solution dominates the current global best solution, the global best solution is updated; when the new Pareto solution

dominates the current local best solution, the local best solution is updated; when the new solution and the current global best solution are mutually non-dominating, the new solution is added to the Pareto set; and when the new solution does not meet any of the above criteria, it is accepted according to the Metropolis acceptance function (Karimi-Mamaghan et al., 2023).

4.1. States and actions in the proposed Q-ALNS

In the proposed Q-ALNS, Q-learning is utilized to select perturbation or local search operators at each iteration, as illustrated in Algorithm 2. The set of actions is correspondingly defined to determine which operator to apply, represented as Eq. (36), where a refers to a candidate action, with either p or (d,r) is selected, and the total number of operators is $|A|$. Each local search operator is associated with a set of destroy and repair operations. Upon selecting an action, the corresponding operations are performed on the current solution χ, leading to a new solution χ'.

$$A = \{1, 2, \dots, a\} \tag{36}$$

In Q-learning, a state represents the condition or situation of the environment at a particular time. It is a critical factor in deciding the action to be taken to achieve the optimal solution. The state is determined by evaluating whether the current solution shows any improvement, thus indicating whether the algorithm is trapped in a local optimum. Therefore, in the Q-learning mechanism of this study, the state is set as a binary set $S = \{0, 1\}$. If new state $s' = 0$, it implies that the search in the latest iteration could not find a better solution. Conversely, if $s' = 1$, it means that a better solution was found after the latest iteration.

In each iteration of the Q-ALNS, the performance of the operators is evaluated, and the reward is calculated accordingly. The Q-table is then updated based on this reward and the current s, as Eq. (37). Finally, using the ϵ-greedy strategy, the next operator is selected based on the current state and the updated Q-table.

$$Q(s,a) = Q(s,a) + \alpha\left(reward + \gamma \max_{a'} Q(s',a') - Q(s,a)\right) \tag{37}$$

4.2. Reward function in the proposed Q-ALNS

In Q-learning-based AOS, Q-value updates replace the traditional weight updates, where operators were previously assigned scores, and now they are assigned rewards (Boualamia et al., 2023; Johnn et al., 2023). Therefore, careful design of the reward function is crucial, as it determines the feedback each operator receives and directly influences the learning process. In each iteration of the Q-ALNS, a perturbation or local search operator is selected, and the corresponding operation is performed. Thus, the evaluation of the performance of an operator is conducted at the end of each iteration. The reward can be defined in various ways. In this work, we reference the definition proposed by Karimi-Mamaghan et al. (2023) to avoid the algorithm getting stuck in local optima and accelerate convergence. In their work, the reward is a function of global and local improvements of the solutions. Extending their methods, we incorporate the consideration of iteration count to privilege improvements at later iterations, as illustrated in Algorithm 3.

The improvement (Δ_{imp}) in the solution includes enhancements in both global (ΔG) and local (ΔL) optima, computed using Eqs. (38) and (39), respectively. These improvements are combined linearly in Eq. (40), and multiplied by the iteration count to calculate the reward (see Eq. (41)), where e is the iteration count and C is a normalization constant. Due to the difference in magnitude between iteration counts and improvement values, the result is normalized by dividing by a constant. A penalty mechanism was also incorporated: if no improvement is obtained, the reward is adjusted to includethe opportunity cost OC, reflecting the maximum prior improvement of unselected operators, as described in Eq. (42).

$$\Delta G = \max\left(\frac{R^* - R^*_{prev}}{R^*_{prev}}, 0\right) \tag{38}$$

$$\Delta L = \max\left(\frac{R - R_{prev}}{R_{prev}}, 0\right) \tag{39}$$

$$\Delta imp = \Delta G \times \eta + \Delta L \times (1 - \eta) \tag{40}$$

$$reward = \frac{\Delta imp \times e}{C} \tag{41}$$

$$reward = \frac{(\Delta imp - \mathrm{OC}) \times e}{C} \tag{42}$$

5. Experimental design

5.1. Experimental setting

The study scenario in this research is based on an unmanned warehouse B, located in Linyi City, Shandong Province, China. This warehouse primarily serves to consolidate and store regional less-than-truckload (LTL) cargo. Fig. 3 illustrates a typical operational scenario within the unmanned warehouse and provides a visualization of the warehouse layout using simulation. The facility faces significant daily demand fluctuations, with morning hours dominated by inbound trucks delivering e-commerce parcels and afternoon peaks requiring intensive outbound shipments. Traditional exclusive dock assignments result in poor resource utilization during demand troughs while causing truck queues during peaks. Although our observations are based on this specific case, the conflict between fixed docking resources and fluctuating demand is a widespread issue in logistics and warehousing environments (Neagoe et al., 2021; Wolff et al., 2021). Warehouse B features docks on both sides for loading and unloading. Storage positions within the warehouse are allocated based on proximity to these docks, with the goal of storing goods as close as possible to the loading and unloading areas. Although Warehouse B has an I-shaped layout, its operational flow more closely resembles a U-shape, making it well-suited for using MSM docks (Rijal et al., 2019). In this layout and service mode, the docks at both ends are not designated exclusively for unloading or loading; their functions are determined based on the demand structure, which aligns well with on-site management needs.

We conducted an on-site survey at Warehouse B to collect detailed operational data, including the layout of docks (e.g., the number and coordinates of docks), truck dispatch records (e.g., truck types, pallet quantities and time window), cargo storage and retrieval plans (e.g., shelf positions for each pallet type), AGV operation speeds, and the switching time for MSM docks. Based on this data, we generated ten sets of test instances, as shown in Table 3. In each instance, every truck has a predefined type, arrival and expected departure time window, total cargo volume, and the quantity of each cargo type. The positions of the docks and shelves are represented by coordinates, with the corresponding shelf number for each cargo type known. Following this setup, we design a three-phase experiment.

In Phase 1, we conduct the operator filtering (OF) experiments to identify better-performing operator combinations. The goal is to pinpoint the operators that exhibit superior performance. In the Q-ALNS, the outer loop consists of several perturbation operators, and the inner loop includes a set of local search operators. Given our two-layer loop structure, the experiment is designed to first compare the performance of local search operators, addressing the question, “Which local search operators perform better?” After this, we combine the better-performing local search operators with perturbation operators to determine “Which perturbation operators perform better?”

In Phase 2, we use a set of benchmarks to validate and evaluate the competitiveness of the proposed Q-ALNS, as outlined in Table 4. ALNS_i refers to the original ALNS, with i indicating different combina-

Algorithm 1: Pseudo-code for ALNS with Q-learning (Q-ALNS).

Input: P ; // Set of perturbation operators
Input: OP ; // Set of local search operators
Input: A ; // Action space, $P \cup OP = A$
Input: $\epsilon \in \mathbb{R}$; // Epsilon-greedy parameter
Input: $\epsilon_0 \in \mathbb{R}$; // Epsilon-greedy initialization parameter
Input: $\beta \in \mathbb{R}$; // Epsilon-decay
Input: $\alpha \in \mathbb{R}$; // Learning rate
Input: $\gamma \in \mathbb{R}$; // Discount factor
Input: $E \in \mathbb{N}$; // Size of iteration
Input: $N \in \mathbb{N}$; // Size of local search loop
Input: $l \in \mathbb{N}$; // Size of learning loop
Input: $\eta \in \mathbb{R}$; // Global improvement weight
Output: χ^* ; // Optimized solution found

Function ALNS($\epsilon, \beta, \alpha, \gamma, E, N, \eta, A$)**:**
  $\chi \leftarrow$ initialsolution() ; // Construct initial solution
  $\chi^* \leftarrow \chi$; // Initialize best solution
  $T \leftarrow -\sum f_k(\text{sol})/\log(0.5)$; // Initialize temperature
  $Q \leftarrow [0]$; // Initialize Q-table
  $s \leftarrow 0$; // Initialize state
  $a \leftarrow$ randomchoice(A) ; // Initialize action
  $\epsilon \leftarrow \epsilon_0$; // Initialize ϵ
  // Main loop
  for $e \leftarrow 1$ **to** E **do**
    $R_{\text{prev}} \leftarrow R_{\min}(\chi)$; // Remember current local optimum before a local search episode
    $R^*_{\text{prev}} \leftarrow R_{\min}(\chi^*)$; // Remember the best solution found before a local search episode
    $R \leftarrow R_{\text{prev}}$; // Remember the best local optimum found during an episode
    $R^* \leftarrow R^*_{\text{prev}}$; // Remember the best solution found during an episode
    // Learning loop
    if $e < l$ **then**
      $a \leftarrow$ randomchoice(A) ; // During a learning loop action is drawn randomly from A
    // Perturbation-local search loop
    else
      $a \leftarrow$ Q-learning($R_{\text{prev}}, R^*_{\text{prev}}, R, R^*, \epsilon, \beta, \alpha, \gamma, e, \eta, A, s, P, OP$) ; // Action based on Q-table
    // Perturbation phase
    if $a \in P$ *(Perturbation operator selected)* **then**
      $\chi' \leftarrow P(\chi, a)$; // Do perturbation
    // Local search phase
    else if $a \in OP$ **then**
      for $t \leftarrow 1$ **to** N **do**
        $\chi' \leftarrow L(\chi, a)$; // Local search (destroy-repair)
    // Pareto and acceptance updating phase
    // Update best solution found
    if *Pareto dominate*(χ', χ^*) **then**
      $\chi^* \leftarrow \chi'$
      $R^* \leftarrow R_{\min}(\chi^*)$
    else if *Pareto dominate*(χ', χ) **then**
      $\chi \leftarrow \chi'$
      $R \leftarrow R_{\min}(\chi)$
    else if *mutually non-dominating*(χ', χ^*) **then**
      add χ' to Pareto set
    else if *accept*(χ', T) **then**
      $\chi \leftarrow \chi'$
    update T ; // Update temperature
    update Q, s ; // Update Q-table
  return χ^* ; // Return the best solution

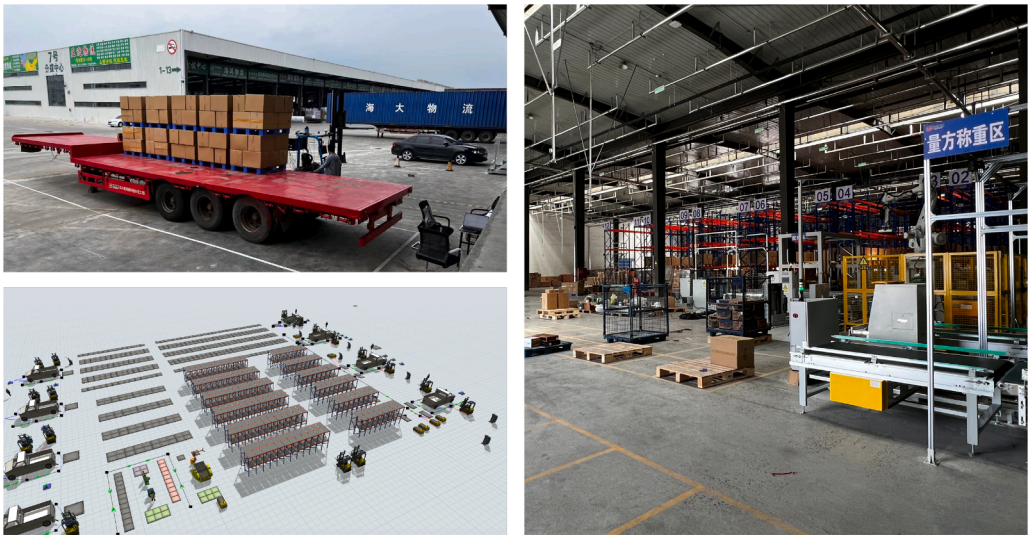


Fig. 3. Layout and typical operational scenario of warehouse B.

Algorithm 2: Pseudo-code for Q-learning Function.

Input: R_{prev}, R^*_{prev}, R, R^*, ϵ, β, α, γ, e, η, A, s, P, OP
Output: a' ; // Next action a'

Function Q-learning($R_{prev}, R^*_{prev}, R, R^*, \epsilon, \beta, \alpha, \gamma, e, \eta, A, s, P, OP$):
- // Calculate reward based on equations
- $reward \leftarrow \text{CalculateReward}(R_{\text{prev}}, R^*_{\text{prev}}, R, R^*, e, \eta)$;
- // Determine state based on improvement during the iteration
- **if** $R^* < R^*_{prev}$ **then**
  - $s' \leftarrow 1$;
- **else**
  - $s' \leftarrow 0$;
- // Update Q-table
- $Q(s,a) \leftarrow Q(s,a) + \alpha\big(reward + \gamma \max_{a'} Q(s',a') - Q(s,a)\big)$;
- // Apply epsilon-decay strategy to move from exploration to exploitation gradually
- $\epsilon \leftarrow \epsilon \cdot \beta$;
- // Choose an action using ϵ-greedy strategy
- **if** $rand() \geq \epsilon$ **then**
  - $a' \leftarrow \text{argmax}_{a''} Q(s', a'')$;
- **else**
  - $a' \leftarrow \text{randomchoice}(A)$;
- **return** a'

Algorithm 3: Q-learning Reward Calculation with Penalty Mechanism.

Input: R^*, R^*_{prev}, R, R_{prev}, η, e, C
Output: $reward$; // Reward

Function CalculateReward($R^*, R^*_{prev}, R, R_{prev}, e, \eta$):
- // Calculate Global and Local Improvements
- $\Delta G \leftarrow \max\left(\frac{R^* - R^*_{prev}}{R^*_{prev}}, 0\right)$;
- $\Delta L \leftarrow \max\left(\frac{R - R_{prev}}{R_{prev}}, 0\right)$;
- // Calculate Weighted Improvement
- $\Delta imp \leftarrow \Delta G \cdot \eta + \Delta L \cdot (1 - \eta)$;
- $\text{OC} \leftarrow \max(\Delta imp)$;
- // Check for Improvement
- **if** $\Delta G > 0$ *or* $\Delta L > 0$ **then**
  - $reward \leftarrow \frac{\Delta imp \times e}{C}$;
- **else**
  - $reward \leftarrow \frac{(\Delta imp - \text{OC}) \times e}{C}$; // Include opportunity cost if no improvement
- **return** $reward$

tions of operators based on the number of better-performing operators identified. RLNS replaces the AOS mechanism in the ALNS framework with a random selection mechanism. S-ALNS is a commonly used score-based ALNS in the literature (Bodnar et al., 2017; Rijal et al., 2019). Q-ALNS-L is a learning mechanism that focuses solely on selecting the local search operator, similar to mechanisms in the current literature (Boualamia et al., 2023; Wang et al., 2024). The first four benchmarks in Table 4 follow the classical perturbation-then-local search structure. Gurobi 11.0.1 is also included as a commercial solver for comparison. The evaluation comprehensively covers multiple aspects: efficiency and accuracy through comparison with benchmarks, multi-objective optimization capabilities via analysis of Pareto fronts, and adaptability to various problem instances. We also examine the impact of different parameter settings, action set size, and reward functions on the algorithm's performance. These analyses ensure a thorough understanding of the strengths of the Q-ALNS.

In Phase 3, we conduct a comparative analysis of different dock mode arrangement strategies to evaluate the effectiveness of the proposed approach. In most existing studies, the number and positions of MSM docks are predefined. To achieve greater flexibility, this paper introduces an adaptive dock mode decision strategy, which jointly decides the MSM dock arrangement, truck assignment, and scheduling. To validate the advantages of this strategy, we compare it against two representative predefined approaches: the exclusive mode, where each dock is fixed for either loading or unloading (widely used in practice), and the fully mixed mode, where all docks are predefined as MSM. The partial mix mode, which requires manual specification of the proportion and positions of MSM docks, is excluded from the comparison due to its scenario dependence and potential configuration bias. We refer to the three strategies compared as Adaptive (this paper), Fixed, and Mixed strategies.

Table 3
Set of test instances for TASP-DMD.

Instance	Number of Docks	Number of trucks	Instance	Number of docks	Number of Trucks
1	3	20	6	8	100
2	4	30	7	8	130
3	5	50	8	9	160
4	6	60	9	9	180
5	7	70	10	10	200

Table 4
Comparison benchmarks for the Q-ALNS.

Index	Benchmark algorithm	Comparison purpose
1	ALNS_i	Effectiveness of multiple operator combinations against a single operator combination
2	RLNS	Effectiveness of Q-learning-based AOS than random operator selection
3	S-ALNS	Effectiveness of Q-learning-based AOS than score-based AOS
4	Q-ALNS-L	Effectiveness of flexible perturbation and local search structure than fixed perturbation-then-local search structure
5	Gurobi	Validate and evaluate the optimality gap

Table 5
Comparison Metrics used in the experiments.

Phase	Metric	Index	Purpose
1	Dominate ratio	Z_n	Quality
2	Relative percentage deviation	RPD	Accuracy
2	Time taken to reach the best solution	CPU time	Efficiency
2	Nondominance Ratio	NR	Quality
2	Hypervolume	HV	Diversity
2	Hierarchical cluster counting	HCC	Uniformity
3	Relative percentage deviation	RPD	Accuracy
3	Time taken to reach the best solution	CPU time	Efficiency

5.2. Performance comparison metrics

Each experiment we discussed in Section 6, involving various instances, operators, and mechanisms, was independently run 10 times. For each run, we recorded the mean, the best value found for every objective, and CPU time. The comparison metrics used in the experiment are shown in Table 5.

In Phase 1, we used a dominance ratio metric to quantify the performance advantage of each local search and perturbation operator in our experiments. The dominance ratio Z_n for a given operator n is defined as the proportion of pairwise comparisons where n shows a statistically significant performance improvement over other operators. The result of this test is denoted as $\zeta_{n,n'}$, where:

$$\zeta_{n,n'} = \begin{cases} 1 & \text{if } n \text{ significantly outperforms } n', \\ -1 & \text{if } n \text{ is significantly outperformed by } n', \\ 0 & \text{if there is no significant difference between } n \text{ and } n'. \end{cases} \tag{43}$$

Next, we use Eq. (44) to count the number of times each operator n wins in the comparison. Here, m represents the total number of operators, and $\delta(\cdot)$ is the indicator function, which equals 1 if the condition inside is true and 0 otherwise. The dominance ratio Z_n is then calculated as the ratio of the count C_n to the total number of pairwise comparisons, using Eq. (45).

$$C_n = \sum_{n'=1, n' \neq n}^{m} \delta(\zeta_{n,n'} = 1) \tag{44}$$

$$Z_n = \frac{C_n}{m} \tag{45}$$

In Phase 2, we evaluate the quality of the solutions using two types of methods. We first focus on three key metrics: relative percentage deviation (RPD) of the mean and best values, and the average computational time (CPU time) required to obtain the best solution. Lower values in these metrics indicate better performance of operators and selection mechanisms. As the tardiness objective is a shared concern for both trucks and warehouses, the RPD calculation is based on the tardiness. The formula for RPD (%) is provided in Eq. (46), where R_i represents the tardiness value of experiment i, and R_{best} is the best objective value (tardiness) found among all experiments.

$$\text{RPD}_i = \frac{(R_i - R_{\text{best}})}{R_{\text{best}}} \times 100 \tag{46}$$

To evaluate the ability to find Pareto fronts, we employ three metrics: nondomination ratio (NR) (Tan et al., 2001), hypervolume (HV) (Zitzler & Thiele, 1999), and hierarchical cluster counting (HCC) (Guimaraes et al., 2009). The NR measures the proportion of solutions in the combined Pareto front that are non-dominated. Let P_B and P_C be the Pareto solution sets obtained by two different algorithms B and C, respectively. The NR is calculated as Eq. (47), where $|\cdot|$ denotes the cardinality of the set. A higher NR value indicates that algorithm B contributes more useful candidate solutions to the combined Pareto front, reflecting its superior performance.

$$NR_B = \frac{|\{\mathbf{x} \in P_B \mid \mathbf{x} \text{ is nondominated in } P_{B \cup C}|}{|P_{B \cup C}|} \tag{47}$$

The formula for HV is provided in Eq. (48), where S represents the set of Pareto solutions, $(f_1, \ldots, f_m)$ are the objective functions, and $(f_1^*, \ldots, f_m^*)$ is the reference point. This metric calculates the volume of the objective space that is dominated by the Pareto front and bounded by the reference point. It provides a measure of both the convergence and diversity of the solutions.

$$HV(S) = \text{volume}\left(\bigcup_{x \in S} [f_1(x), f_1^*] \times \cdots \times [f_m(x), f_m^*] \right) \tag{48}$$

The HCC metric assesses the diversity of solutions by performing hierarchical clustering and counting the resulting clusters. The process involves applying a clustering algorithm to group the solutions and then summing the distances between these clusters. The formula for HCC is provided in Eq. (49), where k represents the number of clusters and d_i denotes the distances between them. This metric offers insights into the distribution and variety of solutions within the Pareto front.

$$HCC = \sum_{i=1}^{k-1} d_i \tag{49}$$

In Phase 3, we continue to use RPD and CPU time as metrics for comparison. Furthermore, all statistical comparisons are conducted using the Wilcoxon signed-rank test (Wilcoxon, 1992). This step is essential to ensure the stability and reliability of the results across different instances and run times, enabling us to validate whether the observed differences in performance are statistically significant. This analysis confirms the robustness of the proposed algorithm across various scenarios.

Table 6
Parameters of the Q-ALNS and tuned values obtained through Grid-search.

Parameter	Notation	Parameter Values			Tuned value
		Low	Medium	High	
Temperature scale	θ	0.01	0.105	0.2	0.2
Epsilon-greedy initialization	ϵ_0	0.7	0.85	1	1
Epsilon-greedy decay	β	0.99	0.995	1	0.99
Learning rate	α	0.5	0.75	1	0.5
Discount factor	γ	0.7	0.85	1	0.7
Local search iteration	ι	100	200	300	100
Local/global improvement weight	η	0.6	0.7	0.8	0.8
Learning loop	l	50	100	200	50

5.3. Parameters tuning

Our algorithm includes eight key parameters: θ, ϵ_0, β, α, γ, ι, η, and l, as shown in Table 6. For parameter tuning, we used the grid search method.

6. Numerical and statistical results

This section executes the three-phase experiment outlined in Section 5 to validate the effectiveness and competitiveness of the Q-ALNS. All experiments were conducted using Python 3.8 on the Snellius Cluster High-Performance Computer, specifically on an AMD Rome 7H12 with CPUs running at 2.6 GHz with 32GB of RAM. Unless otherwise stated, the comparison results have passed significance tests.

6.1. Phase 1: comparison between operators

In this phase, we identify the most advantageous operators for the problem through a series of experiments, assessing their impact on search quality and efficiency.

Common ranking methods in the literature for comparing operator performance include frequency-based, ablation-based, and pairwise comparison matrices. Given the multiple operators in the Q-ALNS, we employ the pairwise comparison metric method using the Wilcoxon test. This approach involves comparing operators in pairs and filling a pairwise comparison matrix based on statistically significant performance differences, resulting in a dominance matrix for all operators.

The local search operators in the Q-ALNS consist of destroy and repair operators, as listed in Table 7. These operators are selectively combined into specific combinations, each tailored to the objective functions (see Table 8). The sources and detailed descriptions are provided in Appendix A. For the perturbation phase of the algorithm, we design three types of perturbation operators tailored for unloading-only (I), loading-only (O), and mixed-mode (F) docks, as shown in Table 9.

In the OF experiments, we evaluate the performance of local search operators in the inner loop. Each local search operator op_i is independently tested over 10 instances with 100 iterations each, and each experiment is repeated 10 times to ensure reliability. We set a dominance ratio threshold of 40% to identify better-performing local search operators. Operators that outperform 40% of others in pairwise comparisons are considered superior. With three prioritized objective functions $f_1 > f_2 > f_3$, operators are filtered accordingly. The filtering process, illustrated in Fig. 4, identify operators $op2$, $op9$, and $op11$ as the better-performing local search operators. Additionally, we compare the performance of perturbation operators in the outer loop over 30 iterations and find that P_1 significantly outperforms P_0 and P_2.

From the OF experiments, we reduce the original set of 16 local search operators to 3 and identify a single effective perturbation operator. Fig. 5 compares three different operator sets: all operators without filtering ('ALL'), all perturbation operators combined with the three selected local search operators ('3P + 3op'), and the selected perturbation operator combined with the three selected local search operators ('P + 3op'). As shown, the operator combination after filtering ('P + 3op') consistently outperforms the other two groups in terms of tardiness, distance, and makespan. Moreover, the filtered operators ('P + 3op') demonstrate higher stability, with a moderate increase in CPU time. The performance benefits clearly justify the effectiveness of the operator filtering procedure. Detailed benefits of the OF process are discussed in Section 6.2.

6.2. Phase 2: performance analysis of the Q-ALNS against benchmark algorithms

In this stage of the experiment, we first compare three ALNS_i mechanisms with RLNS to assess the impact of better-performing operator combinations on the exploration-exploitation capability of the ALNS. Next, we compare the Q-ALNS with RLNS and the commonly used S-ALNS to demonstrate the effectiveness of the Q-learning-based operator selection mechanism compared to random and score-based selection mechanisms.

6.2.1. Comparison based on optimality gap and CPU time

Table 10 presents the performance of three ALNS_i algorithms, RLNS, S-ALNS, Q-ALNS-L and the Q-ALNS, across 10 instances. Since we filtered three local search operators, the ALNS_i algorithm has three versions: ALNS_1, ALNS_2, and ALNS_3. The "AvS" column shows the average RPD (ARPD) of each algorithm from 10 independent runs per instance, while the "BS" column displays the RPD of the best solution. The "T(s)" column indicates the average CPU time. Fig. 6(a) shows the boxplots of RPD for tardiness and CPU time, respectively.

Comparison of ALNS_1, ALNS_2, ALNS_3, and RLNS data shows RLNS performs better in both "AvS" and "BS" (lower values). Fig. 6(a) shows RLNS with a lower mean (black dots) and minimum value, and a smaller interquartile range, indicating better robustness. This indicates that the use of multiple operators, as seen in RLNS, significantly enhances the competitiveness of ALNS in solving the TASP-DMD problem, compared to relying on individual operators. While all algorithms utilize the best operators, RLNS benefits from combining multiple operators, leading to improved performance.

The Q-ALNS achieves better AvS and BS values across all instances. The "Average" row in Table 10 indicates that the integration of Q-learning significantly improves solution quality. Compared to RLNS, the ARPD drops from 18.2% to 6.1%, and the best RPD drops from 12.7% to 0.4%. Compared to S-ALNS, ARPD decreases from 18.1% to 6.1%, and best RPD from 12.3% to 0.4%. According to the average CPU time reported in Table 10 and the CPU time distribution illustrated in Fig. 6(a), the introduction of the Q-learning mechanism (Q-ALNS) results in minimal overhead complexity compared to the random mechanism (RLNS). In most instances, the CPU time remains similar, with an average CPU time difference of merely 1.09 seconds. However, compared to the commonly employed score-based mechanism (S-ALNS), integrating Q-learning significantly enhances computational efficiency, reducing the average CPU time by approximately 38.6 seconds. Furthermore, compared to Q-ALNS-L, which uses the fixed perturbation-then-local search structure, Q-ALNS reduces the ARPD from 16.6% to 6.1%, and the best RPD from 9.8% to 0.4%. These results suggest that our flexible Q-learning mechanism–capable of selecting both perturbation and local search operators–effectively balances exploration and exploitation, outperforming existing approaches with fixed searching structures.

We further compared the convergence behavior of the algorithms. Fig. 6(b) shows the convergence curves of all algorithms for representative cases 3_20 and 10_200. The Q-ALNS demonstrates a lower convergence curve with consistent behavior during the search process. It shows faster initial convergence, as evidenced by the rapid decline in its convergence curve during early iterations, and achieves lower final convergence values, reflecting its capability to find better final solutions. The convergence curves of Q-ALNS are smoother and exhibit less fluctu-

Table 7
Description of local search operators.

ID	Index	Destroy or repair operation
1	rRd	Randomly remove a truck from all available trucks
2	rMxTar	Remove the truck causing the most significant delay
3	rMxM	Remove the truck with the greatest weighted cargo handling distance
4	iBck	Swap the removed truck with its preceding truck at the same dock
5	iFwd	Swap the removed truck with its succeeding truck at the same dock
6	iSwap	Swap the removed truck with another randomly selected unloading or loading truck
7	iUp	Reallocate the removed truck to a dock with a higher rank based on handling distance
8	iDown	Reallocate the removed truck to a dock with a lower rank based on handling distance
9	iDockInsert	Insert the removed truck at all positions in the truck sequence of the same dock
10	iBtwInsert	Insert the removed truck randomly in the truck sequence of different docks
11	riInD2D	Swap trucks between two unloading docks
12	riOuD2D	Swap trucks between two loading docks
13	riFlxD2D	Swap trucks between two mixed-mode docks

Table 8
The combination of local search operators used in the Q-ALNS.

ID	Operators	Target	ID	Operators	Target
op_1	rRd & iBck	Sequence Adjustment	op_9	rMxM & iUp	Sequence & Dock Adjustment
op_2	rRd & iFwd	Sequence Adjustment	op_{10}	rRd & iDockInsert	Sequence & Dock Adjustment
op_3	rRd & iSwap	Sequence & Dock Adjustment	op_{11}	rRd & iBtwInsert	Sequence & Dock Adjustment
op_4	rRd & iUp	Dock Adjustment	op_{12}	rMxTar & iDockInsert	Sequence & Dock Adjustment
op_5	rRd & iDown	Dock Adjustment	op_{13}	rMxTar & iBtwInsert	Sequence & Dock Adjustment
op_6	rMxTar & iBck	Sequence Adjustment	op_{14}	riInD2D	Dock Adjustment
op_7	rMxTar & iSwap	Distance & Dock Adjustment	op_{15}	riOuD2D	Dock Adjustment
op_8	rMxM & iSwap	Distance & Dock Adjustment	op_{16}	riFlxD2D	Dock Adjustment

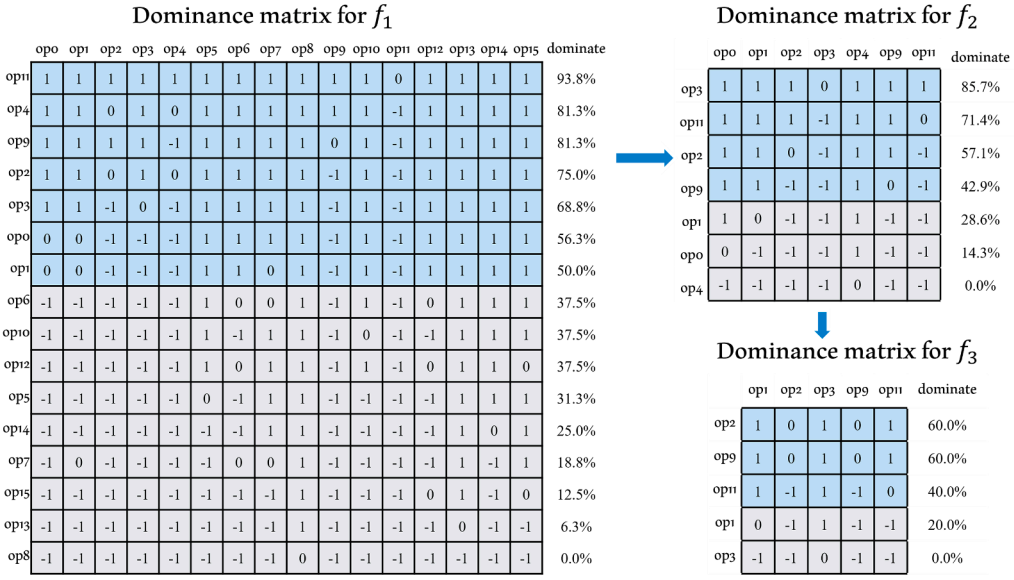


Fig. 4. Local search operator filtering based on the Wilcoxon dominance matrix.

Table 9
Description of perturbation operators.

ID	Index	Perturbation operation
P_1	Chn2I	Randomly selects a dock and changes its type to unloading-only (I)
P_2	Chn2F	Randomly selects a dock and changes its type to mixed-mode (F)
P_3	Chn2O	Randomly selects a dock and changes its type to loading-only (O)

ation, suggesting a more stable search process and reduced interference from local optima.

This phase of experiments demonstrates the successful identification of better-performing operators suitable for the TASP-DMD problem. Integrating the Q-learning mechanism significantly enhances the algorithm's exploration of the solution space. Meanwhile, the commonly used score-based AOS mechanism exhibits limited improvement in solution quality (as evidenced by its comparison with RLNS) and introduces considerable computational overhead. This observation aligns with the discussions presented by Voigt (2024), emphasizing the necessity for optimizing the existing AOS framework. Moreover, the benefits of merely applying the learning mechanism within the classical perturbation-then-local search structure are limited, as evidenced by its comparison with RLNS and Q-ALNS. Conversely, our carefully designed Q-learning mechanism adaptively selects perturbation and local search operators throughout the search process, exhibiting superior performance across the majority of tested scenarios. These findings confirm that the Q-learning-based AOS mechanism not only substantially improves solution quality but also ensures a more robust and efficient search process.

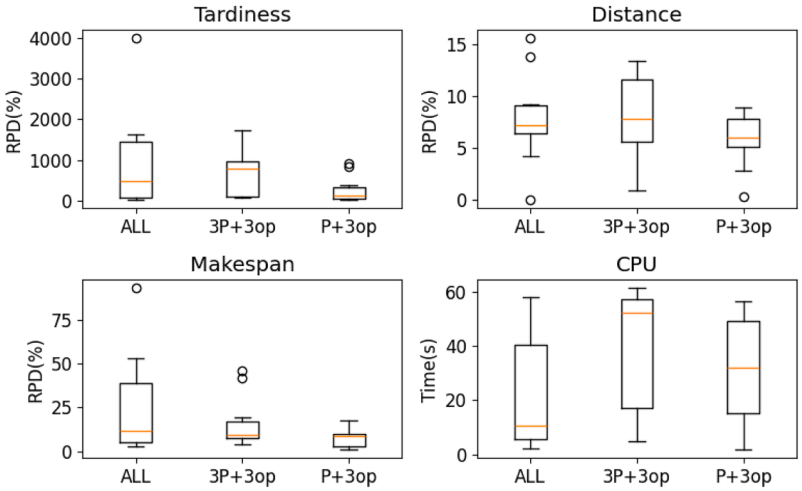
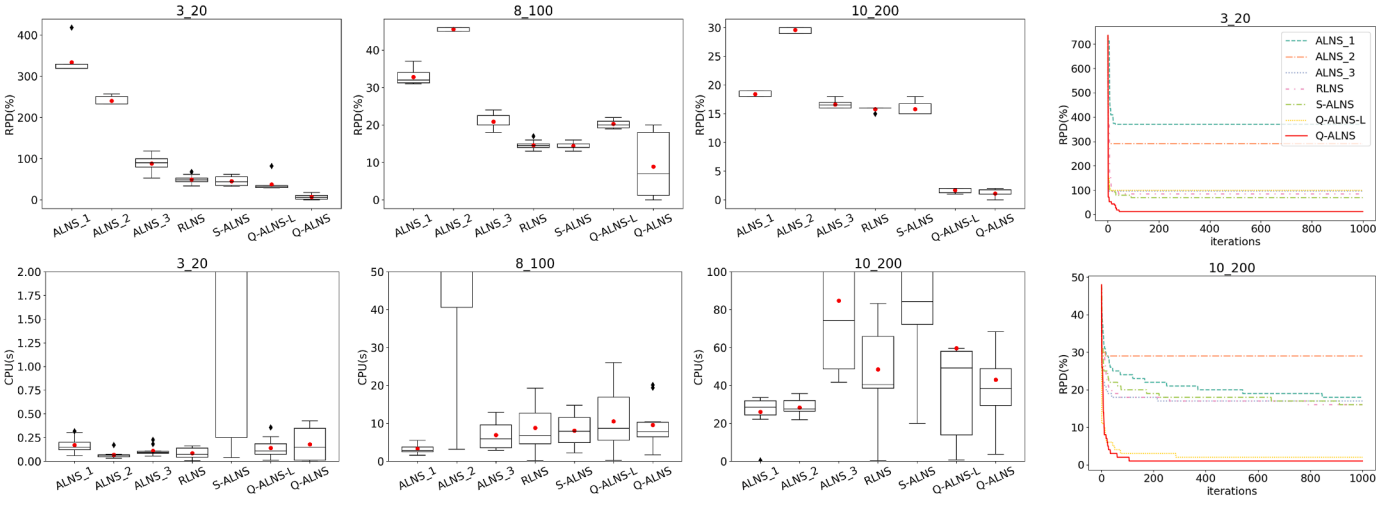


Fig. 5. Comparison of three types of operators sets for the instance with 7 docks and 70 trucks.

Table 10
Overall comparison of the Q-ALNS and classical ALNS.

Instance set		ALNS_1			ALNS_2			ALNS_3			RLNS			S-ALNS			Q-ALNS-L			Q-ALNS		
Dock	Truck	AvS(%)	BS(%)	T(s)	AvS(%)	BS(%)	T(s)	AvS(%)	BS(%)	T(s)	AvS(%)	BS(%)	T(s)	AvS(%)	BS(%)	T(s)	AvS(%)	BS(%)	T(s)	AvS(%)	BS(%)	T(s)
3	20	334.3	319.1	0.2	240.5	233.0	0.1	88.1	52.8	0.1	49.5	33.8	0.1	45.7	33.3	19.7	45.5	16.7	0.1	**7.6**	**0.0**	0.2
4	30	71.6	64.6	0.1	89.0	89.0	0.4	69.5	64.0	0.1	37.2	29.1	0.1	35.0	27.3	10.4	36.7	25.4	0.2	**9.2**	**0.0**	0.3
5	50	42.5	34.9	0.5	59.9	56.1	2.2	41.7	29.9	2.9	15.0	9.0	1.3	14.1	6.4	11.0	12.9	8.4	1.1	**6.9**	**0.0**	1.2
6	60	14.7	10.1	1.0	30.1	29.0	6.2	28.9	21.5	16.8	15.5	8.6	5.4	15.6	11.8	46.9	13.5	8.6	1.1	**9.4**	**0.0**	7.4
7	70	738.6	723.8	3.9	783.7	764.3	5.0	72.5	35.0	5.0	18.6	4.3	4.0	23.8	2.6	29.5	12.2	0.0	2.5	**7.1**	0.6	2.5
8	100	33.0	31.3	3.3	45.5	44.8	58.5	20.8	17.9	6.9	14.5	13.0	8.8	14.3	12.7	8.1	14.6	12.5	10.5	**8.9**	**0.0**	9.5
8	130	8.1	6.9	2.2	17.0	16.1	51.7	8.6	7.3	34.4	5.3	4.3	18.4	5.2	3.9	79.2	5.3	3.9	27.2	**5.0**	**1.4**	24.8
9	160	10.3	9.5	3.0	18.4	18.1	86.3	7.0	6.2	59.9	5.3	4.5	34.7	5.8	5.0	92.7	5.3	4.1	77.0	**1.0**	**0.0**	29.4
9	180	8.0	6.9	4.3	17.8	17.1	149.4	7.2	5.7	102.5	5.4	4.7	47.2	5.7	4.9	137.2	4.8	3.9	122.4	5.2	**1.7**	61.0
10	200	18.4	17.9	26.0	29.6	29.2	28.4	16.8	16.0	84.7	15.8	15.1	48.5	15.8	14.7	130.2	15.3	14.4	59.6	**1.2**	**0.0**	43.0
Average		127.9	122.5	4.4	133.2	129.7	38.8	36.1	25.6	31.3	18.2	12.7	16.8	18.1	12.3	56.5	16.6	9.8	30.2	**6.1**	**0.4**	17.9



(a) Boxplot based on ARPD of tardiness and CPU

(b) Convergence rate

Fig. 6. Comparison of the Q-ALNS, RLNS, S-ALNS, and ALNS_i based on different metrics.

6.2.2. *Comparison based on Pareto front*

This section compares the Q-ALNS with Gurobi and RLNS to demonstrate its ability to find (near-) optimal Pareto fronts. Gurobi solutions are obtained using the ϵ-constraints method, in which f_2 and f_3 are relaxed into constraints. Then the model is solved using Gurobi 11.0.1 with Python 3.8.

Gurobi failed to find feasible solutions within the time limit of 3600 seconds as the problem size increased. Therefore, only results from three small-scale instances are presented. As shown in Table 11, the Q-ALNS achieves a higher proportion of non-dominated solutions (NR) on average (51%) compared to Gurobi (48%), demonstrating better quality in finding Pareto-optimal solutions. It also outperforms Gurobi in HV,

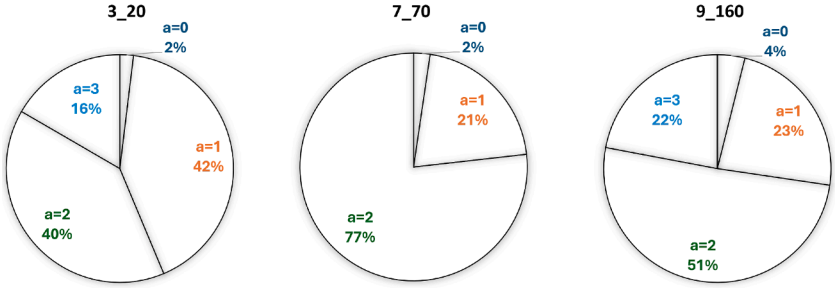


Fig. 7. Average improvement performance of each operator in certain instances.

Table 11
Overall comparison of Q-ALNS with Gurobi based on Pareto front.

Instance		NR		HV		HCC	
Dock	Truck	Gurobi	Q-ALNS	Gurobi	Q-ALNS	Gurobi	Q-ALNS
3	20	**1**	0	**0.844**	0.478	1.012	**3.000**
4	30	0.25	**0.75**	0	**0.818**	0.859	**3.610**
5	50	0.22	**0.78**	0	**0.304**	0.366	**1.263**
Average		0.49	0.51	0.281	0.533	0.746	2.624

Table 12
Overall comparison of the Q-ALNS with RLNS based on Pareto front.

Instance		NR		HV		HCC	
Dock	Truck	RLNS	Q-ALNS	RLNS	Q-ALNS	RLNS	Q-ALNS
3	20	0.302	**0.698**	**0.645**	0.419	3.192	**7.562**
4	30	0.160	**0.840**	0.715	0.758	4.304	**11.231**
5	50	0.569	0.431	0.036	**0.937**	6.243	**14.079**
6	60	0.294	**0.706**	**0.454**	0.334	7.542	**12.093**
7	70	0.431	**0.569**	0.231	**0.513**	4.525	**9.122**
8	100	0.379	**0.621**	0.545	0.551	7.234	**15.568**
8	130	0.123	**0.877**	0.254	**0.588**	4.025	**5.698**
9	160	0.227	**0.773**	0.264	**0.379**	2.187	**5.955**
9	180	0.078	**0.922**	0.345	**0.664**	4.018	**7.629**
10	200	0.264	**0.736**	0.163	**0.335**	2.761	**7.020**
Average		0.283	0.717	0.365	0.548	4.603	9.596

with an average HV of 0.533 compared to Gurobi's 0.281, indicating a more comprehensive exploration of the solution space. Additionally, Q-ALNS performs better in HCC, with an average HCC of 2.624 compared to Gurobi's 0.746, indicating better solution uniformity. Overall, the Q-ALNS is comparable to or better than Gurobi, validating its effectiveness in solving the TASP-DMD problem.

For larger instances, the Q-ALNS is compared with RLNS, as shown in Table 12. The Q-ALNS demonstrates superior performance over RLNS in most instances. It achieves a higher average NR value (0.717) compared to RLNS (0.283), indicating greater effectiveness in producing non-dominated solutions. In terms of HV, it averages 0.548, outperforming RLNS's 0.365. For HCC, the Q-ALNS performs better, with an average HCC of 9.596 compared to RLNS's 4.603, suggesting more evenly distributed solutions.

6.2.3. Adaptiveness of the Q-ALNS to problem instances

This section analyzes how the Q-ALNS adaptively selects perturbation and local search operators based on problem characteristics. We use the relative improvement (RI) index (Karimi-Mamaghan et al., 2023) to evaluate the contribution of different local search operators throughout the search process. RI is calculated by Eq. (50), where $R(a)$ represents the average improvement per application of local search operator a over the entire search process. $R(a)$ is determined by Eq. (51), where $GI(a)$ represents the total global improvement contributed by operator a and $S(a)$ represents the number of improvements achieved by operator a. $RI(a)$ then expresses this average improvement as a percentage of the total improvement achieved by all local search operators.

$$RI(a) = \frac{R(a)}{\sum_{a' \in A} R(a')} \times 100\% \tag{50}$$

$$R(a) = \frac{GI(a)}{S(a)} \tag{51}$$

Fig. 7 shows the average RI results for three randomly selected instances with varying characteristics, through 10 times run. The pie charts illustrate the distribution of the RI among three local search operators ($a = 0, a = 1, a = 2$) and one perturbation operator ($a = 3$) across three instances (3_20, 7_70, and 9_160). The results show clear variations in each operator's contribution depending on the instance characteristics. For example, in instance 3_20, operator $a = 1$ and $a = 2$ dominate the process, whereas the perturbation operator ($a = 3$) plays a minor role. In instance 7_70, operator $a = 2$ dominates with 77%. As problem size increases, such as in instances 9_160, the perturbation operator ($a = 3$) again demonstrates an increased contribution. These results demonstrate the Q-ALNS's ability to adaptively select the most suitable operators according to problem features, efficiently handling a wide range of problem scales and complexities. Although variations occur in the selection frequency of operators across individual runs, the Q-ALNS's consistent performance in terms of solution quality and stability indicates that these variations have minimal impact on overall algorithm effectiveness.

Fig. 8 provides a detailed visualization of operator selection throughout the search process for typical instances 7_70 and 9_160. Different colored bars represent the selected operator for each iteration (green for $a = 0$, orange for $a = 1$, purple for $a = 2$, and gray for $a = 3$), while the black solid line represents the convergence curve. Notably, the zoomed-in subplots highlight iterations 50–200, as the first 50 iterations involve random selection before the Q-learning mechanism is activated. This allows the Q-learning agent to initialize reward feedback and state-action values. The charts show the scale-adaptive optimization behaviors during the search process. For the instance 7_70, the algorithm achieves rapid improvement in the early stage, followed by a clear shift toward repeated use of operator $a = 0$ for intensive local search, and operator $a = 3$ for occasional perturbation to maintain diversification. This indicates that the learned policy quickly identified effective operators for local improvement, while still maintaining a degree of perturbation to prevent premature convergence. In contrast, instance 9_160 exhibits a slower initial improvement and a more balanced operator selection pattern throughout. After the random phase, all four operators are actively and frequently selected, suggesting that the learning mechanism maintains a higher degree of exploration. This behavior reflects the algorithm's attempt to discover effective combinations across a more complex and larger search space. These observations confirm the Q-ALNS's ability to align its search strategy with both problem structure and optimization progress, achieving an effective balance between exploration and exploitation across varying problem scales.

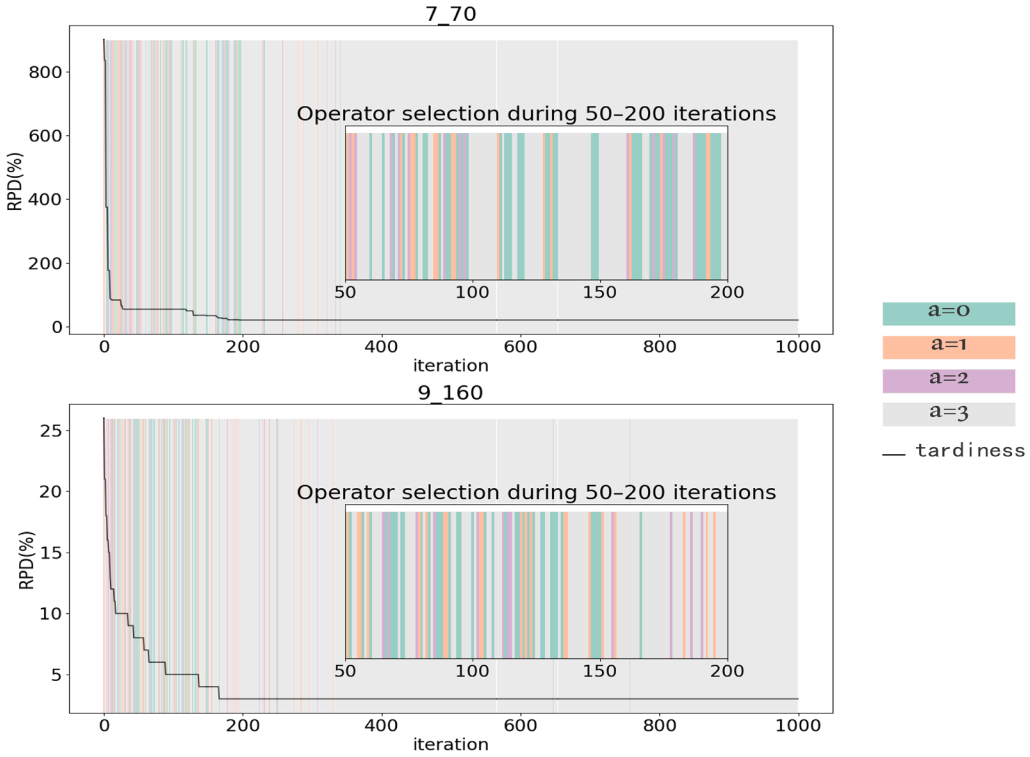


Fig. 8. Selection of operators at each iteration in the search process for certain instances.

Table 13
Comparison of different action space configurations within the Q-learning-based AOS mechanism.

Instance		19-pl-action space			3-ls-action space			4-pl-action space (Q-ALNS)		
Dock	Truck	AvS(%)	BS(%)	T(s)	AvS(%)	BS(%)	T(s)	AvS(%)	BS(%)	T(s)
3	20	20.7	2.5	0.1	45.5	16.7	0.1	**7.6**	**0.0**	0.2
4	30	52.5	3.1	0.4	36.7	25.4	0.2	**9.2**	**0.0**	0.3
5	50	17.7	0.4	1.3	12.9	8.4	1.1	**6.9**	**0.0**	1.2
6	60	13.5	1.3	10.4	13.5	8.6	1.1	**9.4**	**0.0**	7.4
7	70	35.0	4.9	1.7	12.2	0.0	2.5	**7.1**	**0.6**	2.5
8	100	18.2	2.4	19.1	14.6	12.5	10.5	**8.9**	**0.0**	9.5
8	130	5.5	2.2	37.6	5.3	3.9	27.2	**5.0**	**1.4**	24.8
9	160	3.6	1.4	52.3	5.3	4.1	77.0	**1.0**	**0.0**	29.4
9	180	6.3	4.5	72.6	4.8	3.9	122.4	5.2	**1.7**	61.0
10	200	3.0	2.0	81.4	15.3	14.4	59.6	**1.2**	**0.0**	43.0
Average		17.6	2.5	27.7	16.6	9.8	30.2	**6.1**	**0.4**	17.9

Table 14
Comparison of Q-ALNS with a 0–1 reward mechanism.

Instance		Original 0–1 reward mechanism			Value-based reward mechanism (Q-ALNS)		
Dock	Truck	AvS(%)	BS(%)	T(s)	AvS(%)	BS(%)	T(s)
3	20	68.6	18.7	0.1	**7.6**	**0.0**	0.2
4	30	7.3	23.2	0.1	**9.2**	**0.0**	0.3
5	50	4.3	12.7	6.0	**6.9**	**0.0**	1.2
6	60	22.3	9.9	54.7	**9.4**	**0.0**	7.4
7	70	28.5	18.8	3.6	7.1	**0.6**	2.5
8	100	11.8	8.1	7.7	**8.9**	**0.0**	9.5
8	130	10.8	4.4	14.8	**5.0**	**1.4**	24.8
9	160	5.2	4.6	16.5	**1.0**	**0.0**	29.4
9	180	7.4	4.4	42.3	**5.2**	**1.7**	61.0
10	200	14.8	3.1	30.3	**1.2**	**0.0**	43.0
Average		18.1	10.8	17.6	**6.1**	**0.4**	17.9

Table 15
Performance comparison of Fixed, Mixed, and Adaptive (our) dock mode strategies, with relative improvements. The values reported are the best-performing solutions across 10 runs for each strategy. Tard. and Span are measured in hours, Dist. in meters. Relative improvements are calculated row-wise.

Instance	Fixed				Mixed				Adaptive			
	Tard.(h)	Dist.(m)	Span(h)	Util.	Tard.(h)	Dist.(m)	Span(h)	Util.	Tard.(h)	Dist.(m)	Span(h)	Util.
3_20	2.6	1231.8	6.3	66.3%	1.8	1119.5	6.3	73.5%	1.6	1170.1	6.2	71.0%
4_30	15.4	2340.6	6.9	70.2%	14.3	2518.2	6.4	89.3%	8.9	2401.5	6.3	77.4%
5_50	20.9	2918.1	7.3	80.1%	30.4	2780.0	7.2	93.6%	9.6	2374.4	6.3	92.6%
6_60	39.8	3840.7	7.0	87.3%	60.0	3898.1	7.3	95.1%	41.0	3957.5	6.6	93.3%
7_70	17.0	3738.9	8.1	60.6%	4.8	3756.2	7.2	80.5%	4.9	3796.2	7.2	75.8%
8_100	181.5	7498.5	10.7	74.2%	205.5	7810.3	9.8	95.5%	156.0	7688.5	8.9	80.2%
8_130	296.7	10414.8	11.5	94.8%	374.1	10085.2	12.3	96.8%	289.4	10035.7	11.0	96.0%
9_160	497.9	13319.9	12.5	96.8%	620.6	13497.5	14.0	96.4%	494.0	13022.8	12.6	94.8%
9_180	593.5	15048.1	14.6	91.6%	763.1	15331.1	15.7	95.4%	587.3	14721.2	13.8	97.3%
10_200	731.1	17464.8	15.9	86.4%	917.3	18035.9	16.5	95.7%	702.7	17097.3	14.2	92.3%
Avg	239.6	7781.6	10.1	80.8%	299.2	7883.2	10.3	91.2%	229.5	7626.5	9.3	87.0%
Instance	Mixed vs Fixed (%)				Adaptive vs Fixed (%)				Adaptive vs Mixed (%)			
	Tard.	Dist.	Span	Util.	Tard.	Dist.	Span	Util.	Tard.	Dist.	Span	Util.
3_20	-31.6%	-9.1%	0.1%	10.8%	-38.3%	-5.0%	-1.2%	7.0%	-9.8%	4.5%	-1.3%	-3.4%
4_30	-7.1%	7.6%	-8.1%	27.3%	-42.5%	2.6%	-8.7%	10.3%	-38.0%	-4.6%	-0.7%	-13.4%
5_50	45.6%	-4.7%	-1.1%	16.8%	-53.9%	-18.6%	-13.4%	15.7%	-68.4%	-14.6%	-12.4%	-1.0%
6_60	50.8%	1.5%	3.7%	9.0%	3.2%	3.0%	-6.1%	6.9%	-31.6%	1.5%	-9.5%	-1.9%
7_70	-71.7%	0.5%	-11.1%	32.9%	-71.2%	1.5%	-10.2%	25.1%	1.7%	1.1%	1.0%	-5.9%
8_100	13.2%	4.2%	-8.4%	28.7%	-14.1%	2.5%	-16.3%	8.0%	-24.1%	-1.6%	-8.6%	-16.1%
8_130	26.1%	-3.2%	7.1%	2.1%	-2.4%	-3.6%	-4.3%	1.3%	-22.6%	-0.5%	-10.6%	-0.8%
9_160	24.7%	1.3%	11.8%	-0.4%	-0.8%	-2.2%	0.8%	-2.1%	-20.4%	-3.5%	-9.8%	-1.6%
9_180	28.6%	1.9%	7.6%	4.1%	-1.0%	-2.2%	-5.5%	6.1%	-23.0%	-4.0%	-12.2%	1.9%
10_200	25.5%	3.3%	3.6%	10.8%	-3.9%	-2.1%	-10.9%	6.8%	-23.4%	-5.2%	-13.9%	-3.6%
Avg	10.4%	0.3%	0.5%	14.2%	-22.5%	-2.4%	-7.6%	8.5%	-26.0%	-2.7%	-7.8%	-4.6%

Note: The proposed scheme operates on a daily planning basis, with a 24-hour work cycle, in alignment with the context of automated warehousing. While tardiness exceeds 24 hours in certain instances, this is due to the cumulative delay costs of all vehicles. On average, the delay per vehicle does not exceed 4 hours. Furthermore, as indicated by the 'Span' column, the maximum operational completion time is no more than 15 hours, ensuring that all inbound and outbound tasks are completed within the same day.

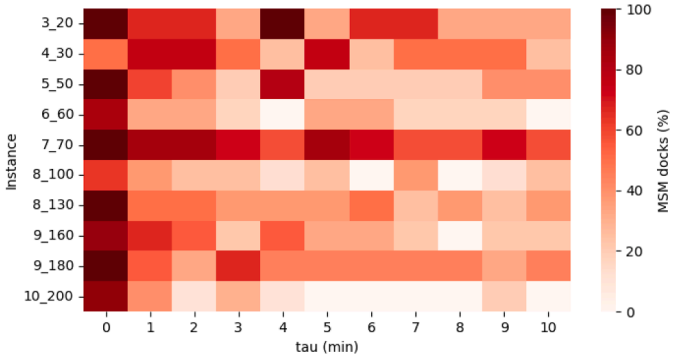


Fig. 9. Effect of mode switching time on proportion of MSM docks.

6.2.4. Sensitivity analysis

This section conducts a sensitivity analysis of the Q-ALNS's action space, reward mechanism, and switching time to assess how variations in these components affect the solution's performance and robustness.

Action set size

The action space in our algorithm is determined by the selected operators. Specifically, we evaluate three configurations to analyze the impact of action space size and the inclusion of perturbation operators in the Q-learning-based AOS mechanism. The first configuration (4-pl-action space) consists of one perturbation operator and three local search operators, which is the one we use in Q-ALNS. The second configuration (19-pl-action space) expands the action space to three perturbation operators and 16 local search operators, allowing us to assess whether a filtering process leads to better performance. The third configuration (3-ls-action space) removes the perturbation operator from the Q-learning mechanism and instead applies a fixed perturbation after each local search step, allowing us to evaluate the necessity of adaptive perturbation in the AOS framework.

Table 13 presents the comparison results based on three metrics. The 4-pl-action space configuration achieves lower Avs, better BS, and faster CPU time compared to the other two configurations. The superior performance over the 3-ls-action space confirms that replacing fixed perturbation with an adaptive approach significantly enhances algorithm efficiency. In our case study, it leads to an average solution quality improvement of over 10.5 percentage points while reducing CPU time by 12.3s. Similarly, the improvement over the 19-pl-action space demonstrates that our OF process selects efficient operators tailored to the problem, ensuring both solution quality and search efficiency. This results in an average solution quality improvement of over 11.5 percentage points and a CPU time reduction of 9.8s.

These findings highlight the effectiveness of OF process and emphasize the benefit of including perturbation operators within the Q-learning mechanism. An appropriately balanced action space, combining both perturbation and targeted local search operators, is essential for maintaining a robust trade-off between exploration and exploitation, ultimately leading to consistently better performance.

Reward function

This section analyzes the impact of the reward mechanism on the performance of the Q-ALNS. We compare the original 0–1 reward mechanism with the proposed value-based reward mechanism (detailed in Section 4.2) to understand how different reward structures affect the algorithm's performance. The results in Table 14 reveal that the value-based reward mechanism significantly outperforms the original 0–1 reward mechanism across most test instances. Specifically, the proposed mechanism consistently achieves lower average AvS values and superior BS metrics, indicating more stable performance and better solution

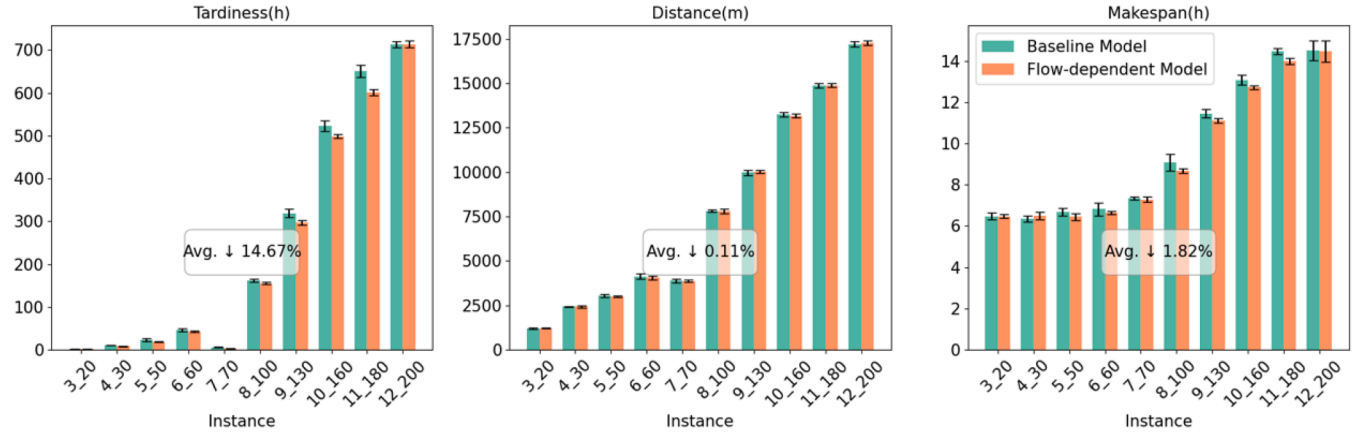


Fig. 10. Comparison of original and extended models in terms of tardiness, internal transport distance and makespan, with mean and standard deviation across 10 runs.

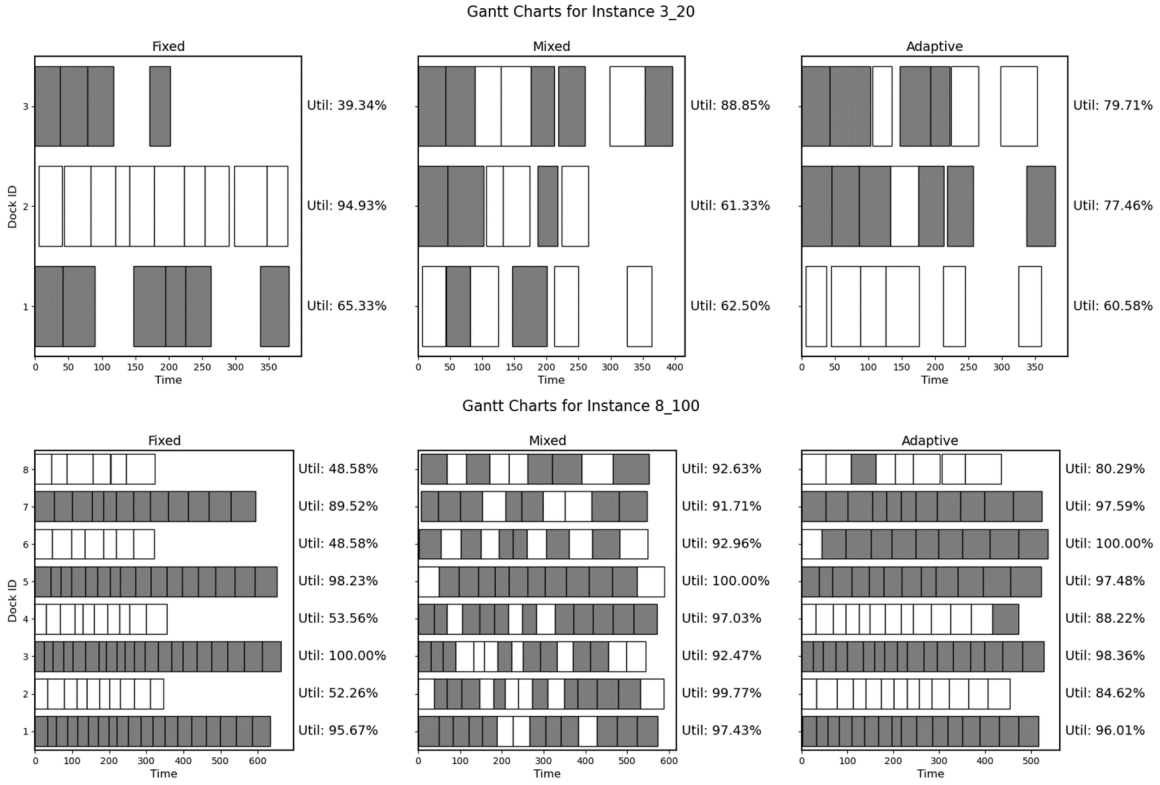


Fig. 11. Gantt charts of the Fixed, Mixed, and Adaptive strategy for two instances. The time dimension is measured in minutes.

quality. Notably, in several larger instances, the value-based mechanism found optimal solutions (BS equal or close to zero) more frequently than the original mechanism. Additionally, the CPU time comparison highlights that the original 0–1 reward mechanism may sometimes require substantially more computational effort, particularly in mid-size instances. In contrast, the carefully designed value-based reward mechanism efficiently guides the algorithm toward promising search regions. These findings highlight the effectiveness of the value-based reward mechanism. While the original 0–1 reward mechanism achieves acceptable performance, the value-based reward mechanism offers a better balance between solution quality and computational effort.

Switching time

This section explores the influence of dock mode switching time. We model the dock reconfiguration time as a constant "switching cost" to capture the overhead of reconfiguring MSM docks. Based on monitoring data from warehouse B (in Section 5.1), we observed that the reaction time for task switching at MSM docks is approximately 5 minutes on average. Fig. 9 shows how increasing the switching time from 0 to 10 minutes reduces the percentage of MSM docks selected, especially for larger instances. This indicates that in very busy systems, the benefits of MSM docks can be limited by higher switching costs. Nevertheless, for most problem sizes, MSM docks remain advantageous when the switching time is moderate (e.g., under five minutes).

We further compared the baseline model, which applies a constant setup time τ at MSM docks, with the flow-dependent model (presented in Appendix B), where τ is only incurred when consecutive trucks differ in commodity type. Fig. 10 shows that the flow-dependent model reduces average tardiness per truck by about 14.67%, while distance and makespan remain nearly unchanged. This indicates that the baseline

assumption already provides robust performance. As warehousing systems mature–exhibiting higher levels of digitalization and operational flexibility–the flow-dependent model can better exploit these dynamics and attain superior performance.

6.3. Phase 3: comparison of dock mode decision with pre-set scenarios

In this phase, we compare three dock operation strategies: Adaptive, Fixed, and Mixed. Table 15 reports the best-performing solutions (with the lowest tardiness) for each instance under the three strategies. For each strategy, the first three metrics–Tardiness (Tard.), Distance (Dist.), and Makespan (Span)–represent the actual objective values. The fourth column indicates the dock utilization rate. For the Fixed and Mixed modes, the same parameterized Q-ALNS algorithm was used, excluding the perturbation phase and employing predefined dock configurations.

The Adaptive strategy outperforms both Fixed and Mixed in terms of operational efficiency. On average across the ten instances, it reduces total system-wide tardiness by 22.5% (10.1 hours per day per instance) and makespan by 7.6% (0.8 hours per day per instance), while maintaining a relatively low internal transport distance compared to the Fixed strategy. This demonstrates that introducing a moderate proportion of MSM docks significantly reduces tardiness and makespan while avoiding excessive complexity. Mixed strategy tends to increase tardiness in certain instances, which aligns with prior research (Bodnar et al., 2017; Rijal et al., 2019) indicating diminishing returns from too much flexibility. By selectively incorporating MSM docks rather than converting all docks, managers can better balance internal transport distances against timeliness objectives.

Beyond merely deciding the proportion of MSM docks, our integrated framework adaptively configures the functional roles of all docks. Fig. 11 illustrates how the Adaptive strategy arranges dock functionalities based on anticipated demand. In small-scale cases, MSM functionality is assigned to docks 2 and 3, while in large-scale cases, it shifts to docks 4, 6, and 8. These decisions are not based on fixed patterns (e.g., centralized or evenly spaced layouts) as in previous studies, but are instead optimized in response to the task distribution. This dynamic configuration improves overall efficiency by optimizing not only the placement of MSM docks, but also the assignment of dedicated loading-only and unloading-only docks, offering a practical tool for operators to align resource allocation with real-time demand.

In practice, flexible dock reconfiguration often relies on both physical infrastructure and managerial agility. Previous work often acknowledged but did not model these complexities, potentially overestimating the benefits of MSM docks. By including switching times, our model offers more realistic guidance for operators. Sensitivity analysis in Section 6.2.4 indicates that while switching costs can slightly offset the benefits of flexibility, the Adaptive strategy remains robust under reasonable switching durations. The proposed adaptive strategy has been embedded in warehouse B (in Section 5.1) management system to plan daily inbound and outbound operations based on forecasted demands. Since the CPU time for most cases is under one minute, this approach can also be re-run if unexpected demand fluctuations occur. Future research will explore more stochastic and dynamic scenarios to further extend the model's applicability.

7. Conclusion

This paper investigates the integrated truck assignment and scheduling problem with dock mode decision, extending the flexibility of mixed service mode (MSM) docks compared to the commonly used pre-defined dock mode configurations in existing research. We constructed a mathematical model that integrates truck assignment, scheduling, and dock mode decisions, aiming to minimize tardiness, makespan, and AGV handling distance. To solve this complex combinatorial optimization problem, we proposed a Q-learning-based Adaptive Large Neighborhood Search (Q-ALNS) algorithm, in which Q-learning is embedded to guide the adaptive operator selection (AOS) mechanism. Unlike existing methods that employ Q-learning solely for operator selection within a fixed perturbation-then-local-search framework, our algorithm integrates Q-learning into the search structure itself, enabling dynamic control over both operator selection and the sequencing of perturbation and local search phases.

To validate the proposed approach, we conducted a three-phase experiment. These experiments indicate that the performance of the ALNS relies heavily on filtering better-performing operator combinations, which can significantly enhance the algorithm's adaptability to the problem. The Q-ALNS, which incorporates Q-learning, consistently outperforms the original ALNS and other benchmarks in terms of optimality gap and Pareto front quality while maintaining a competitive time to reach the best solution. Specifically, the Q-learning-based AOS mechanism dynamically adjusts the search strategy according to the problem size and optimization process, consistently demonstrating its effectiveness in balancing exploration and exploitation. Sensitivity analysis further revealed that a well-balanced action space and an effective reward mechanism are crucial for optimizing the performance of the Q-ALNS. These results highlight the structural advantage of our algorithm, where the integration of Q-learning into the search process enables dynamic coordination of operator selection and search phase transitions, thereby enhancing overall adaptability and search efficiency. In our case, selectively introducing MSM docks substantially improves inbound and outbound efficiency, validating the effectiveness of integrating dock mode decisions with truck assignment and scheduling. The proposed approach adaptively assigns dock functionalities based on demand, enabling efficiency-oriented evaluation of MSM dock deployment during facility design and supporting demand-driven reconfiguration of dock modes and truck scheduling in daily planning, thereby serving as a practical decision-support tool for logistics operators.

Despite the significant benefits achieved, the current model assumes a level of certainty in operational conditions, which might not fully capture the complexities of real-world scenarios. A promising direction for future research is to extend the model to account for uncertainty, enhancing its applicability to more dynamic and complex environments. On the algorithmic side, the integration of Q-learning into the ALNS algorithm has proven effective, and future work could enhance the intelligence of parameter selection–for instance, by incorporating operator runtime into the learning process to better balance exploration efficiency and computational cost.

CRediT authorship contribution statement

Yueyi Li: Writing – original draft, Visualization, Software, Methodology, Investigation, Formal analysis, Conceptualization; **Mehrdad Mohammadi:** Writing – review & editing, Validation, Supervision, Methodology, Conceptualization; **Xiaodong Zhang:** Writing – review & editing, Validation; **Yunxing Lan:** Software, Conceptualization; **Willem van Jaarsveld:** Writing – review & editing, Validation, Supervision.

Acknowledgment

Funding: This work was supported by the Natural Science Foundation of China [grant number U2469209]; the National Key Research and Development Program of China [grant number 2021YFB1407003]; and the China Scholarship Council [grant number 202307090057].

Appendix A. Detailed descriptions and sources of local search operators

The local search operators in ALNS are formed by a combination of destroy, repair, and comprehensive operators. Some operators were adapted from the design ideas in Bodnar et al. (2017), Rijal et al. (2019), but were modified to suit the specifics of our problem.

Our design of destroy operators follows the most commonly used random and greedy operators found in the ALNS literature. The random destruction operator randomly selects a subset of elements from the current solution to remove, creating diversity in the search process. The greedy destruction operator, on the other hand, targets the removal of elements that are likely to lead to immediate improvements in the solution quality when reinserted. The destroy operators in the proposed algorithm are as follows:

- **rRd**: Randomly selecting and removing a truck from all available trucks.
- **rMxTar**: Targeting and removing the truck that causes the most significant delay in the current solution.
- **rMxM**: Targeting and removing the truck with the greatest weighted cargo handling distance from all trucks.

The repair strategies considered in this paper include forward, backward, swapping, and inserting. The repair operators are as follows:

- **iBck**: Swap the removed truck with its preceding truck at the same dock.
- **iFwd**: Swap the removed truck with its succeeding truck at the same dock.
- **iSwap**: Depending on whether it is for unloading or loading, swap the removed truck with another randomly selected unloading or loading truck, not limited to the same dock.
- **iUp**: Reallocate the removed truck to a dock with a higher rank based on handling distance.
- **iDown**: Reallocate the removed truck to a dock with a lower rank based on handling distance.
- **iDockInsert**: Attempt to insert the removed truck at all positions in the truck sequence of the same dock, replacing the current solution if improved.
- **iBtwInsert**: Insert the removed truck randomly in the truck sequence of different docks, replacing the current solution if improved.

The comprehensive operators, distinct from the destroy and repair operators, independently modify an existing solution. The designed comprehensive operators in this paper include the following three types:

- **riInD2D**: Randomly selects two docks used for unloading (D_I) and swaps the trucks they service, while maintaining the sequence of trucks.
- **riOuD2D**: Randomly selects two docks used for loading (D_O) and swaps their serviced trucks, keeping the truck order unchanged.
- **riFlxD2D**: In mixed-mode docks (D_F), randomly selects two docks and swaps their serviced trucks, again preserving the truck sequence.

Appendix B. Flow-dependent switching time via auxiliary variables

In our proposed model, the switching time parameter τ acts as a uniform safety margin, representing the minimal reaction time required for a MSM dock to detect whether consecutive trucks are of the same type and to issue any necessary repositioning or task commands. When this reaction time is negligible, the model can be refined by introducing auxiliary binary variables to ensure that τ is applied only when two consecutive trucks differ in type.

We introduce binary variables u_{ijk}, which indicate that truck i immediately precedes truck j on dock k, as defined in Eq. (B.1).

$$u_{ijk} = \begin{cases} 1, & \text{if truck } i \text{ immediately precedes truck } j \text{ on dock } k, \\ 0, & \text{otherwise}, \forall\, i, j \in T,\ k \in D \end{cases} \tag{B.1}$$

These variables satisfy the constraint in Eqs. (B.2) and (B.3). Here $z_{ijk} = 1$ if and only if truck i is served before truck j at dock k.

$$u_{ijk} \le z_{ijk} \tag{B.2}$$

$$\sum_{i \in T} u_{ijk} \le 1, \quad \forall\, j \in T,\ k \in D \tag{B.3}$$

Let $\overline{C}(j)$ denote the set of trucks of the opposite type to j, as in Eq. (B.4).

$$\overline{C}(j) = \begin{cases} O, & j \in I \\ I, & j \in O \end{cases} \tag{B.4}$$

Then processing time H_j is redefined as Eq. (B.5) to apply τ only when consecutive trucks differ in type.

$$H_j = t_j + \frac{1}{v} \sum_{k \in D} m_{jk}\, x_{jk} + \tau \sum_{k \in D} \sum_{i \in \overline{C}(j)} u_{i,j,k} \tag{B.5}$$

Under this extension, Eq. (4) then becomes the Eq. (B.6).

$$f_2 = \min \max_{j \in T} H_j \tag{B.6}$$

Eqs. (B.2) and (B.3) enforce a unique immediate predecessor, equation Eq. (B.5) applies τ only to cross-type transitions, and equation Eq. (B.6) retains the original makespan criterion.

When commodities are further distinguished into multiple categories (e.g., refrigerated, hazardous, regular), the reconfiguration time can be generalized from a single constant τ to a category-pair–dependent parameter $\tau_{c,c'} \ge 0$, where c, c' index commodity categories. Let C denote the set of categories and $c(\cdot) : T \to C$ map each truck to its commodity category. This allows heterogeneous and possibly *asymmetric* setup times (e.g., $\tau_{c,c'} \ne \tau_{c',c}$), reflecting that some transitions (regular → refrigerated) are more time-consuming than others.

Under this extension, Eqs. (B.1)–(B.3) remain unchanged. Eq. (B.4) (which defines the "opposite" set) is no longer needed; instead, we directly index the setup by the predecessor–successor categories. Replace Eq. (B.5) by

$$H_j = t_j + \frac{1}{v} \sum_{k \in D} m_{jk}\, x_{jk} + \sum_{k \in D} \sum_{i \in T} \tau_{c(i),\,c(j)}\, u_{ijk}, \qquad \forall j \in T. \tag{B.5'}$$

Here, $\tau_{c,c}$ can be set to 0 (no reconfiguration when categories match), while $\tau_{c,c'}$ for $c \ne c'$ captures the category-specific switching effort. If desired, dock-specific heterogeneity can also be modeled by $\tau^{(k)}_{c,c'}$ for MSM docks $k \in D_{\text{MSM}}$; this only requires replacing $\tau_{c(i),c(j)}$ by $\tau^{(k)}_{c(i),c(j)}$ in (B.5′). The objective Eq. (B.6) and the predecessor structure (B.1)–(B.3) remain valid. This more general variant goes beyond the scope of the current study and is left for future research.

Specialization, setting $\tau_{c,c} = 0$ and $\tau_{c,c'} = \tau$ for $c \ne c'$ recovers the binary flow-dependent case presented earlier. Setting all $\tau_{c,c'} \equiv \tau$ recovers the constant-buffer baseline.

References

Azadeh, K., De Koster, R., & Roy, D. (2019). Robotized and automated warehouse systems: Review and recent developments. *Transportation Science*, *53*(4), 917–945.

Bakhshi-Khaniki, H., Tavakkoli-Moghaddam, R., Hanzalek, Z., & Vahedi-Nouri, B. (2024). A flexible job shop scheduling problem involving reconfigurable machine tools under industry 5.0. In M. Thürer, R. Riedel, G. von Cieminski, & D. Romero (Eds.), *Advances in production management systems. Production management systems for volatile, uncertain, complex, and ambiguous environments* (pp. 244–257). Cham: Springer Nature Switzerland.

Berghman, L., & Leus, R. (2015). Practical solutions for a dock assignment problem with trailer transportation. *European Journal of Operational Research*, *246*(3), 787–799. https://doi.org/10.1016/j.ejor.2015.05.057

Bodnar, P., de Koster, R., & Azadeh, K. (2017). Scheduling trucks in a cross-dock with mixed service mode dock doors. *Transportation Science*, *51*(1), 112–131. https://doi.org/10.1287/trsc.2015.0612

Boualamia, H., Metrane, A., Hafidi, I., & Mellouli, O. (2023). A new adaptation mechanism of the ALNS algorithm using reinforcement learning. In N. Aboutabit, M. Lazaar, & I. Hafidi (Eds.), *Advances in machine intelligence and computer science applications* (pp. 3–14). Cham: Springer Nature Switzerland (*vol. 656*). https://doi.org/10.1007/978-3-031-29313-9_1

Boysen, N., & Fliedner, M. (2010). Cross dock scheduling: Classification, literature review and research agenda. *Omega*, *38*(6), 413–422. https://doi.org/10.1016/j.omega.2009.10.008

Buijs, P., Vis, I. F. A., & Carlo, H. J. (2014). Synchronization in cross-docking networks: A research classification and framework. *European Journal of Operational Research*, *239*(3), 593–608. https://doi.org/10.1016/j.ejor.2014.03.012

Custodio, L., & Machado, R. (2020). Flexible automated warehouse: A literature review and an innovative framework. *The International Journal of Advanced Manufacturing Technology*, *106*, 533–558.

Fang, W., Zhu, H., & Mei, Y. (2022). Hybrid meta-heuristics for the unrelated parallel machine scheduling problem with setup times. *Knowledge-Based Systems*, *241*, 108193. https://doi.org/10.1016/j.knosys.2022.108193

Guimaraes, F. G., Wanner, E. F., & Takahashi, R. H. C. (2009). A quality metric for multi-objective optimization based on hierarchical clustering techniques. In *2009 IEEE Congress on evolutionary computation* (pp. 3292–3299). Trondheim, Norway: IEEE. https://doi.org/10.1109/CEC.2009.4983362

He, X., & Prabhu, V. V. (2022). Design and analysis of AGV-based cross-docking operations using analytical models. *Production & Manufacturing Research*, *10*(1), 1–20. https://doi.org/10.1080/21693277.2022.2083714

Hermel, D., Hasheminia, H., Adler, N., & Fry, M. J. (2016). A solution framework for the multi-mode resource-constrained cross-dock scheduling problem. *Omega*, *59*, 157–170. https://doi.org/10.1016/j.omega.2015.06.002

Jamili, N., Van Den Berg, P. L., & De Koster, R. (2022). Quantifying the impact of sharing resources in a collaborative warehouse. *European Journal of Operational Research*, *302*(2), 518–529. https://doi.org/10.1016/j.ejor.2022.01.007

JD Corporate Blog (2020). Jd deploys 100 5g-powered agvs at changsha asia no.1. https://jdcorporateblog.com/jd-deploys-100-5g-powered-agvs-at-changsha-asia-no-1/. Accessed: 2025-04-18.

Jiang, Y., Wang, Y., Hu, Z., Xue, Q., & Yu, B. (2023). Airport gate assignment problem with harbor constraints based on branch-and-price algorithm. *Transportation Research Part E: Logistics and Transportation Review*, *176*, 103192.

Johnn, S.-N., Darvariu, V.-A., Handl, J., & Kalcsics, J. (2023). Graph reinforcement learning for operator selection in the alns metaheuristic. In B. Dorronsoro, F. Chicano, G. Danoy, & E.-G. Talbi (Eds.), *Optimization and learning* (pp. 200–212). Cham: Springer Nature Switzerland (*vol. 1824*). https://doi.org/10.1007/978-3-031-34020-8_15

Johnn, S.-N., Darvariu, V.-A., Handl, J., & Kalcsics, J. (2024). A graph reinforcement learning framework for neural adaptive large neighbourhood search. *Computers & Operations Research*, *172*, 106791. https://doi.org/10.1016/j.cor.2024.106791

Karimi Mamaghan, M., Mohammadi, M., Dullaert, W., Vigo, D., & Pirayesh, A. (2024). Dynamic operator management in meta-heuristics using reinforcement learning: An application to permutation flowshop scheduling problems. https://doi.org/10.48550/arXiv.2408.14864

Karimi-Mamaghan, M., Mohammadi, M., Meyer, P., Karimi-Mamaghan, A. M., & Talbi, E.-G. (2022). Machine learning at the service of meta-heuristics for solving combinatorial optimization problems: A state-of-the-art. *European Journal of Operational Research*, *296*(2), 393–422. https://doi.org/10.1016/j.ejor.2021.04.032

Karimi-Mamaghan, M., Mohammadi, M., Pasdeloup, B., & Meyer, P. (2023). Learning to select operators in meta-heuristics: An integration of q-learning into the iterated greedy algorithm for the permutation flowshop scheduling problem. *European Journal of Operational Research*, *304*(3), 1296–1330. https://doi.org/10.1016/j.ejor.2022.03.054

Kembro, J., & Norrman, A. (2022). The transformation from manual to smart warehousing: an exploratory study with swedish retailers. *The International Journal of Logistics Management*, *33*(5), 107–135.

Kim, J., Goo, B., Roh, Y., Lee, C., & Lee, K. (2023). A branch-and-price approach for airport gate assignment problem with chance constraints. *Transportation Research Part B: Methodological*, *168*, 1–26.

Ladier, A.-L., & Alpan, G. (2016). Cross-docking operations: Current research versus industry practice. *Omega*, *62*, 145–162. https://doi.org/10.1016/j.omega.2015.09.006

Majthoub Almoghrabi, M., & Sagnol, G. (2025). Surgery scheduling in flexible operating rooms by using a convex surrogate model of second-stage costs. *European Journal of Operational Research*, *321*(1), 23–40. https://www.sciencedirect.com/science/article/pii/S0377221724005952. https://doi.org/10.1016/j.ejor.2024.07.036

Naderi, B., Roshanaei, V., Begen, M. A., Aleman, D. M., & Urbach, D. R. (2021). Increased surgical capacity without additional resources: Generalized operating room planning and scheduling. *Production and Operations Management*, *30*(8), 2608–2635. https://doi.org/10.1111/poms.13397

Nadi, A., Sharma, S., Snelder, M., Bakri, T., van Lint, H., & Tavasszy, L. (2021). Short-term prediction of outbound truck traffic from the exchange of information in logistics hubs: A case study for the port of rotterdam. *Transportation Research Part C: Emerging Technologies*, *127*, 103111. https://www.sciencedirect.com/science/article/pii/S0968090X21001303. https://doi.org/10.1016/j.trc.2021.103111

Neagoe, M., Hvolby, H.-H., & Turner, P. (2021). Why are we still queuing? exploring landside congestion factors in australian bulk cargo port terminals. *Maritime Transport Research*, *2*, 100036. https://doi.org/10.1016/j.martra.2021.100036

Neamatian Monemi, R., & Gelareh, S. (2023). Dock assignment and truck scheduling problem; consideration of multiple scenarios with resource allocation constraints. *Computers & Operations Research*, *151*, 106074. https://doi.org/10.1016/j.cor.2022.106074

Neamatian Monemi, R., Gelareh, S., & Maculan, N. (2023). A machine learning based branch-cut-and-benders for dock assignment and truck scheduling problem in cross-docks. *Transportation Research Part E: Logistics and Transportation Review*, *178*, 103263. https://doi.org/10.1016/j.tre.2023.103263

Odeyinka, O. F., Omoegun, O. G., Odeyinka, O. F., & Omoegun, O. G. (2023). Warehouse operations: An examination of traditional and automated approaches in supply chain management. In *Operations management - recent advances and new perspectives*. IntechOpen. https://doi.org/10.5772/intechopen.113147

Petri, M., Turi, A., & Mocan, M. (2019). Warehouse redesign challenges–adapting layout and improving process flow. In *International symposium in management innovation for sustainable management and entrepreneurship* (pp. 635–646). Springer.

Pisinger, D., & Ropke, S. (2019). Large neighborhood search. In M. Gendreau, & J.-Y. Potvin (Eds.), *Handbook of metaheuristics* (pp. 99–127). Cham: Springer International Publishing (*vol. 272*). https://doi.org/10.1007/978-3-319-91086-4_4

Rijal, A., Bijvank, M., & de Koster, R. (2019). Integrated scheduling and assignment of trucks at unit-load cross-dock terminals with mixed service mode dock doors. *European Journal of Operational Research*, *278*(3), 752–771. https://doi.org/10.1016/j.ejor.2019.04.028

Rijal, A., Bijvank, M., & de Koster, R. (2023). Dynamics between warehouse operations and vehicle routing. *Production and Operations Management*, *32*(11), 3575–3593. https://doi.org/10.1111/poms.14051

Ropke, S., & Pisinger, D. (2006). An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transportation Science*, *40*(4), 455–472. https://doi.org/10.1287/trsc.1050.0135

Shahmardan, A., & Sajadieh, M. S. (2020). Truck scheduling in a multi-door cross-docking center with partial unloading – reinforcement learning-based simulated annealing approaches. *Computers & Industrial Engineering*, *139*, 106134. https://doi.org/10.1016/j.cie.2019.106134

Shakeri, M., Low, M. Y. H., Turner, S. J., & Lee, E. W. (2012). A robust two-phase heuristic algorithm for the truck scheduling problem in a resource-constrained crossdock. *Computers & Operations Research*, *39*(11), 2564–2577. https://doi.org/10.1016/j.cor.2012.01.002

Shaw, P. (1998). Using constraint programming and local search methods to solve vehicle routing problems. In G. Goos, J. Hartmanis, J. Van Leeuwen, M. Maher, & J.-F. Puget (Eds.), *Principles and practice of constraint programming — CP98* (pp. 417–431). Berlin, Heidelberg: Springer Berlin Heidelberg (*vol. 1520*). https://doi.org/10.1007/3-540-49481-2_30

Sodiya, E. O., Umoga, U. J., Amoo, O. O., & Atadoga, A. (2024). Ai-driven warehouse automation: A comprehensive review of systems. *GSC Advanced Research and Reviews*, *18*(2), 272–282. https://doi.org/10.30574/gscarr.2024.18.2.0063

Stephan, K., & Boysen, N. (2011). Vis-à-vis vs. mixed dock door assignment: A comparison of different cross dock layouts. *Operations Management Research*, *4*(3-4), 150–163. https://doi.org/10.1007/s12063-011-0057-3

Tan, K. C., Lee, T. H., & Khor, E. F. (2001). Evolutionary algorithms for multi-objective optimization: Performance assessments and comparisons. In *Proceedings of the 2001 congress on evolutionary computation (IEEE cat. no.01TH8546)* (pp. 979–986). Seoul, South Korea: IEEE (*vol. 2*). https://doi.org/10.1109/CEC.2001.934296

Turkeš, R., Sörensen, K., & Hvattum, L. M. (2021). Meta-analysis of metaheuristics: Quantifying the effect of adaptiveness in adaptive large neighborhood search. *European Journal of Operational Research*, *292*(2), 423–442. https://doi.org/10.1016/j.ejor.2020.10.045

Tutam, M., & White, J. A. (2019). A multi-dock, unit-load warehouse design. *IISE Transactions*, *51*(3), 232–247. https://doi.org/10.1080/24725854.2018.1488307

Voigt, S. (2024). A review and ranking of operators in adaptive large neighborhood search for vehicle routing problems. *European Journal of Operational Research*, (p. S0377221724003928). https://doi.org/10.1016/j.ejor.2024.05.033

Wang, Q., Wu, P., Guo, L., Chen, P., & Ling, Y. (2024). Improving the ALNS algorithm for PDPTW problem using a DDQN agent. In *2024 IEEE 13th data driven control and learning systems conference (DDCLS)* (pp. 2114–2119). https://doi.org/10.1109/DDCLS61622.2024.10606837

Wilcoxon, F. (1992). Individual comparisons by ranking methods. In *Breakthroughs in statistics: Methodology and distribution* (pp. 196–202). Springer, New York, NY.

Windras Mara, S. T., Norcahyo, R., Jodiawan, P., Lusiantoro, L., & Rifai, A. P. (2022). A survey of adaptive large neighborhood search algorithms and applications. *Computers & Operations Research*, *146*, 105903. https://doi.org/10.1016/j.cor.2022.105903

Wolff, P., Emde, S., & Pfohl, H.-C. (2021). Internal resource requirements: The better performance metric for truck scheduling? *Omega*, *103*, 102431. https://doi.org/10.1016/j.omega.2021.102431

Xi, X., Changchun, L., Yuan, W., & Loo Hay, L. (2020). Two-stage conflict robust optimization models for cross-dock truck scheduling problem under uncertainty. *Transportation Research Part E: Logistics and Transportation Review*, *144*, 102123. https://doi.org/10.1016/j.tre.2020.102123

Xie, F., Li, K., Chen, J., Xiao, W., & Zhou, T. (2025). An adaptive large neighborhood search for unrelated parallel machine scheduling with setup times and delivery times. *Computers & Operations Research*, *177*, 106976. https://doi.org/10.1016/j.cor.2025.106976

Zhang, Y., Negenborn, R. R., & Atasoy, B. (2023). Synchromodal freight transport replanning under service time uncertainty: An online model-assisted reinforcement learning. *Transportation Research Part C: Emerging Technologies*, *156*, 104355. https://doi.org/10.1016/j.trc.2023.104355

Zheng, H., Fan, X., & Tang, C. (2024). Integrated optimization of berth allocation problem with berth shifting strategies and ballast water management for dry bulk ports. *Ocean Engineering*, *309*, 118430. https://doi.org/10.1016/j.oceaneng.2024.118430

Zitzler, E., & Thiele, L. (1999). Multiobjective evolutionary algorithms: A comparative case study and the strength pareto approach. *IEEE Transactions on Evolutionary Computation*, *3*(4), 257–271. https://doi.org/10.1109/4235.797969